



单位代码 10006

学 号 22371240

分 类 号 TP311.5

北京航空航天大学
BEIHANG UNIVERSITY

毕业设计 (论文)

面向深度学习系统模糊测试的测试用例
智能管理与分析

学 院 名 称 计算机学院

专 业 名 称 计算机科学与技术专业

学 生 姓 名 郭俊杰

指 导 教 师 吴际

2026 年 5 月

北京航空航天大学

本科生毕业设计（论文）任务书

I、毕业设计（论文）题目：

面向深度学习系统模糊测试的测试用例智能管理与分析

II、毕业设计（论文）使用的原始资料（数据）及设计技术要求：

1. 查阅深度学习框架测试、模糊测试、向量检索等相关中英文文献。
2. 以 PyTorch 模糊测试场景为背景，分析测试用例、日志与覆盖率数据特点。
3. 设计测试用例管理与分析系统，完成用例存储、检索与分析。
4. 系统采用前后端分离架构，支持候选边界条件分析与结果展示。
5. 论文撰写符合本科毕业设计规范，提交软件、论文及相关材料。

III、毕业设计（论文）工作内容：

1. 完成课题调研与需求分析，明确系统目标与技术路线。
2. 完成测试数据处理流程设计，实现日志解析与特征提取。
3. 实现用例管理与分析功能，包括语义检索、故障辅助定位与候选边界条件分析。
4. 完成系统前后端开发与集成，实现列表展示、详情分析与可视化。
5. 搭建实验环境，基于真实测试数据开展实验并分析结果。
6. 完成论文撰写、修改定稿及答辩准备工作。

IV、主要参考资料：

[1] Pei K, Cao Y, Yang J, et al. DeepXplore:

Automated Whitebox Testing of Deep Learning Systems[C]. SOSP, 2017.

[2] Xie X, Ma L, Juefei-Xu F, et al. DeepHunter:

A Coverage-Guided Fuzz Testing Framework for DNNs[C]. ISSTA, 2019.

[3] Ernst M D, Perkins J H, Guo P J, et al. The Daikon System:

Dynamic Detection of Likely Invariants[J]. Sci. Comput. Program., 2007.

[4] Lewis P, Perez E, Piktus A, et al. Retrieval-Augmented Generation

for Knowledge-Intensive NLP Tasks[C]. NeurIPS, 2020.

计算机 学院 计算机科学与技术 专业类 220611 班

学生 郭俊杰

毕业设计(论文)时间： 2025 年 12 月 15 日至 2026 年 5 月 31 日

答辩时间： 2026 年 6 月 10 日

成 绩： _____

指导教师： _____

兼职教师或答疑教师（并指出所负责部分）：

_____系（教研室）主任（签字）： _____

注：任务书应该附在已完成的毕业设计（论文）的首页。



本人声明

我声明，本论文及其研究工作是由本人在导师指导下独立完成的，在完成论文时所利用的一切资料均已在参考文献中列出。

作者： 郭俊杰

签字：

时间： 2026 年 5 月



面向深度学习系统模糊测试的测试用例智能管理与分析

学 生： 郭俊杰

指导教师： 吴际

摘 要

本论文围绕 PyTorch 深度学习框架模糊测试结果的管理与分析问题展开研究。模糊测试可以生成大量包含边界参数、算子组合和崩溃日志的测试用例，但这些数据在后续使用中仍存在测试产物统一管理不便、用例价值难以直观分析、崩溃故障线索和边界信息不易整合等问题。针对上述问题，论文设计并实现了一套结合语义检索和大语言模型（LLM）的模糊测试数据管理与分析系统。论文主要工作如下：

1. 针对模糊测试产物类型较多、统一管理不便的问题，设计并实现了测试数据管理功能。系统对测试脚本结构、执行状态、覆盖贡献数据、崩溃日志和语义向量进行统一关联，支持测试用例导入、列表查询、条件筛选、详情查看和相似用例召回，并通过结构化元数据和语义向量的协同存储维护测试用例之间的关联关系，为后续数据分析和故障分析提供基础。

2. 针对用例价值分布和补充测试方向难以直观分析的问题，设计并实现了数据分析和用例效能评估功能。系统基于覆盖效能指数（Coverage Efficiency Index, CEI）模型，综合考虑单个测试用例的分支覆盖贡献和加权计算成本，并通过动态百分位阈值筛选低收益用例。系统还在前端展示覆盖缺口、测试用例多样性分布和用例效能排行，并提供低效用例筛选和候选补盲测试脚本生成功能，用于辅助用户了解测试集覆盖情况和用例价值分布。

3. 针对崩溃用例中故障线索和候选边界信息不易整合分析的问题，设计并实现了故障分析功能。系统围绕单个崩溃用例提供故障辅助定位、相似用例查看、候选边界条件提示和边界对比展示等功能。同时，系统基于检索增强生成（Retrieval-Augmented Generation, RAG）方法，整理相似用例、运行信息和候选边界信息，生成辅助诊断报告和候选补盲测试脚本，为用户复查崩溃原因和补充测试提供参考。

关键词：深度学习框架，模糊测试，语义检索，检索增强生成，覆盖效能指数



Intelligent Management and Analysis of Test Cases for Deep Learning System Fuzzing

Author: Guo Junjie

Tutor: Wu Ji

Abstract

This thesis studies the management and analysis of fuzzing outputs for the PyTorch deep learning framework. Fuzz testing can generate many test cases with boundary parameters, unusual operator combinations along with their crash logs. In practice, however, the resulting artifacts cannot be reused directly in later analysis: the heterogeneous outputs are hard to keep under one schema, the value of each case is not obvious to a reader, and crash clues and boundary information are difficult to integrate. To address these problems, this thesis designs and implements a fuzzing data management and analysis system combined with semantic retrieval and large language models (LLMs). The main work is as follows:

1. For data management, the thesis builds a dedicated module that handles the heterogeneous nature of fuzzing artifacts and brings them under a single workflow. The system associates test script structures, execution status, coverage contribution data, crash logs, and semantic vectors in a unified way. It supports test case import, list query, conditional filtering, detail view, and similar-case retrieval, and maintains relations between test cases through the coordinated storage of structured metadata and semantic vectors, providing a basis for later data analysis and fault analysis.

2. For data analysis, data analysis and case efficiency evaluation functions are designed and implemented. The system focuses on case value distribution as well as where additional testing should be added. A Coverage Efficiency Index (CEI) model is introduced. For each case the model balances the branch coverage gained against the weighted computational cost of running it, so that low-benefit cases can be filtered out with a dynamic percentile threshold. On the front end the system displays coverage gaps and the distribution of case diversity, together with rankings of case efficiency. Filtering of low-benefit cases and drafting of gap-filling test



scripts are exposed as separate operations, giving users a clearer picture of coverage status and of how case value is distributed across the suite.

3. For fault analysis, a fault analysis function is designed and implemented to integrate crash clues and candidate boundary information in failed cases. For each crashed case, users can request localization assistance, browse similar historical cases, inspect candidate boundary-condition hints, or compare boundary parameters side by side. It also uses Retrieval-Augmented Generation (RAG) to organize similar cases, runtime information, and candidate boundary information, and generates auxiliary diagnostic reports and candidate gap-filling test scripts. These functions provide references for users to review crash causes and design additional tests.

Key words: Deep Learning Framework, Fuzz Testing, Semantic Retrieval, Retrieval-Augmented Generation, Coverage Efficiency Index



目 录

1 绪论	4
1.1 课题背景	4
1.1.1 深度学习框架的工程应用与测试问题	4
1.1.2 模糊测试的引入与测试数据管理瓶颈	4
1.2 国内外研究现状	6
1.2.1 深度学习框架模糊测试方法研究	6
1.2.2 测试数据管理与效能评估相关研究	7
1.2.3 大语言模型辅助软件测试研究现状	8
1.3 研究目标与主要工作	8
2 相关技术与理论基础	10
2.1 深度学习框架与模糊测试技术	10
2.1.1 深度学习框架的体系结构	10
2.1.2 模糊测试技术原理与分类	10
2.2 文本语义表征与向量检索技术	11
2.2.1 基于孪生网络的预训练语义编码模型	11
2.2.2 高维向量近似最近邻检索	11
2.3 降维分析与不变量推断方法	12
2.3.1 主成分分析原理	12
2.3.2 Daikon 动态不变量检测框架	13
2.4 检索增强生成技术	13
2.5 代码覆盖率采集技术	14
3 系统需求分析与总体设计	15
3.1 系统功能需求分析	15
3.2 系统总体架构设计	16
3.3 系统技术栈选型	17



4 系统详细设计与主要算法	19
4.1 数据接入与特征表示设计	19
4.1.1 基于分层正则匹配的日志解析模块	19
4.1.2 双路向量化特征表示方法	20
4.2 异构存储模块设计	21
4.3 故障辅助分析模块设计	21
4.3.1 基于 DFS 拓扑回溯的故障辅助定位模块	22
4.3.2 语义相似召回与 RAG 辅助诊断	23
4.3.3 故障关联分析	23
4.4 数据分析与优化模块设计	25
4.4.1 基于 PCA 的测试集多样性分析	25
4.4.2 CEI 覆盖效能指数量化模型	25
4.4.3 算子覆盖缺口分析	26
4.4.4 覆盖缺口驱动的候选补盲测试脚本生成	27
5 系统实现与功能展示	29
5.1 开发环境与工具配置	29
5.2 后端主要模块实现	30
5.2.1 Python FastAPI 算法后端实现	30
5.2.2 Java Spring Boot 业务后端实现	31
5.2.3 数据库表结构与向量集合详细设计	32
5.3 前端界面实现与功能展示	33
5.3.1 FuzzHub 运行总览	33
5.3.2 用例管理界面	33
5.3.3 用例详情故障辅助定位界面	35
5.3.4 AI 智能分析助手界面	35
5.3.5 故障关联分析界面	36
5.3.6 算子覆盖缺口分析界面	38
5.3.7 测试效能评估界面	39



5.3.8 测试用例多样性分布界面	40
6 实验与结果分析	41
6.1 实验环境与数据集	41
6.2 物理执行隔离与覆盖率采集设置	42
6.2.1 Gcov/LCOV 探针的编译挂载流程	42
6.2.2 基于 multiprocessing.spawn 的隔离执行模块	43
6.3 CEI 筛选算法的实验分析	45
6.3.1 模拟验证实验	46
6.3.2 真实物理对照实验 (200 例)	47
6.3.3 千例规模物理跑测	48
6.4 Daikon 候选边界条件分析实验	49
6.4.1 已知数学约束的召回实验	49
6.4.2 按约束生成验证样例的实验	50
6.4.3 真实数据中的候选边界条件案例	52
总结与展望	53
致谢	55
参考文献	56



1 绪论

1.1 课题背景

1.1.1 深度学习框架的工程应用与测试问题

深度学习(Deep Learning, DL)框架已经成为人工智能应用开发中比较基础的一层软件。以 PyTorch 为例,开发者通常只接触上层开发接口,底层运行细节则由框架内部处理。这样的设计降低了模型开发门槛,但也意味着一次看起来普通的 API 调用,背后可能要经过框架内部多个环节。当框架版本不断更新、算子数量越来越多时,一些与类型转换、边界维度处理或内存管理相关的问题,靠少量人工样例往往测试不出来 [1]。

在实际测试中,DL 框架的缺陷也不总是直接表现为崩溃。例如某些张量形状组合、广播规则或混合精度设置只在特定执行路径下才会触发异常;另一些问题可能表现为数值结果轻微偏移。这类偏移有时不会立刻抛出运行时错误,单次训练或推理日志中也未必能看到明显提示。如果上层模型长期在包含这类问题的环境中训练,偏差可能被带入权重更新过程,进而影响最终模型行为 [2]。对自动驾驶感知、医疗影像分析等对结果稳定性要求较高的场景,这种影响还可能进一步引起漏检、误判等问题。除数值问题外,底层内存越界、缓冲区溢出等也会影响框架服务的稳定性。包含异常张量形状或边界参数组合的计算图(Computation Graph)有可能触发非法内存访问,从而影响推理服务的可靠性 [3]。

所以 DL 框架测试不能只检查 API 能不能正常调用。同一个算子在不同输入维度、数据类型和参数取值下,可能会进入不同的底层执行路径。当多个算子连接成计算图后,某个算子的边界行为还可能受到前后算子的影响。这就要求测试样例覆盖更多非常规输入,比如极端维度、异常广播关系以及跨算子的参数约束。

1.1.2 模糊测试的引入与测试数据管理瓶颈

针对深度学习框架在较大输入空间下产生的缺陷,传统由人工编写固定输入的单元测试方法存在覆盖不够的问题。框架接口需要同时处理数据类型、张量维度和算子组合等情况,靠少量人工构造的样例很难完全覆盖输入空间。为了解决这个问题,模糊测试(Fuzzing)被引入到 DL 框架测试中 [4]。该方法借助自动化算法构造畸形参数、极端边



界值或不常见的算子组合,将这些输入持续投入框架运行,从而暴露常规样例不容易触发的问题。

具体到 DL 框架,模糊测试产生的输入一般不是简单的字符串或标量值,而是可以直接调用框架 API 的测试脚本。这些脚本中包含计算图结构和参数信息。测试执行之后,系统还会同步产生覆盖、日志和运行状态等相关记录。其中覆盖率数据记录测试用例触达的代码分支或物理行,崩溃日志可能混合多层调用和底层错误信息,运行记录用来说明该用例的运行情况。这些数据一起构成了后续分析的基础,但格式差异较大,分别保存在脚本文件、日志文本和覆盖率报告等位置。

随着模糊测试任务持续运行,这类产物的数量会迅速增多。如果缺少统一的组织方式,用例筛选和故障分析都要较多人工参与。例如,测试脚本要先和对应的覆盖率记录、执行结果以及崩溃日志建立映射关系;同一底层缺陷触发的多条崩溃日志需要被归并;覆盖收益较低但执行成本较高的用例需要被识别出来;历史失效样本里的参数边界和算子组合也需要重新利用起来。所以 DL 框架模糊测试的难点不只在于生成更多输入,还在于如何管理和分析生成之后的大量测试产物。

在崩溃日志检索和归类方面,只靠关键字匹配并不合适。同一底层缺陷可能因为输入脚本、变量名或调用路径不同而产生不同的表层日志;反过来,日志中相似的错误词也可能对应不同算子、不同参数组合或不同后端的问题。传统的文本检索工具主要按字面特征匹配,不太能同时利用算子拓扑结构和历史相似用例等上下文信息。此外,DL 框架的底层运行时封装了大量调度细节,异常堆栈一般停留在运行时或 C++ 内核层,不容易直接对应到应用层中真正触发该异常的算子组合。这就导致开发者需要在测试产物和框架源码之间反复切换,也拉长了问题排查和缺陷复现的周期 [5]。

测试用例的多样性也不能只看脚本文本本身,还需要结合计算图的语义来观察。模糊测试的效果取决于生成算法对输入空间的探索广度,但变异过程在实际运行中常常会反复生成结构相近的算子组合。比如不同脚本在文本上可能差别较大,但计算图拓扑、张量维度变化和参数约束其实非常相似;也可能出现脚本文本相近、却触发不同执行路径的情况。对包含节点属性、边连接关系等信息的计算图,单纯统计算子数量很难反映测试集的真实分布。传统的关系型数据库可以保存元数据,但不擅长表达高维结构之间的相似关系;基础的统计图表也很难展示测试集在语义空间中的聚集、重复和空缺区域 [6]。因此测试数据管理需要引入语义表示和相似检索方法,来支持对历史用例的召回、归类和后续分析。



另外,只看覆盖率也不能完整反映测试用例的实际价值。在模糊测试过程中,有些用例只带来很少的新覆盖,却需要较长的执行时间或较多显存;也有一些用例没有直接触发崩溃,但可能覆盖了之前较少出现的算子组合。如果系统只根据“代码覆盖率”或“是否异常”来判断,就不容易区分哪些用例值得保留,哪些只是在重复消耗资源。另外,覆盖率也说明不了两个用例在计算图结构或参数组合上是否接近,更不能直接给出某类崩溃背后的参数线索。所以本文在后续设计中同时考虑覆盖贡献、运行成本和语义差异,用来筛选低收益的冗余用例,并尝试从历史崩溃样本里整理出可复用的约束信息和补盲方向[7]。本文后续设计主要面向三个问题:如何按语义召回相似用例,如何把低收益用例识别出来,以及如何为崩溃用例生成诊断材料。为此,覆盖效能指数(Coverage Efficiency Index, CEI)被用作衡量覆盖收益的指标;Daikon 约束分析在相似的健康样本与崩溃样本之间寻找候选边界条件;检索增强生成(Retrieval-Augmented Generation, RAG)则负责整理出辅助诊断报告。

1.2 国内外研究现状

1.2.1 深度学习框架模糊测试方法研究

面向 DL 框架的测试研究中,早期工作多数集中在输入生成问题上。Pei 等人提出的 DeepXplore[8]用神经元覆盖率描述模型内部状态,再通过差分测试寻找能够触发不同系统行为的输入样本。这项作为后续的深度系统测试提供了一种比较有代表性的思路。Xie 等人[9]提出的 DeepHunter 进一步利用覆盖反馈指导变异过程,把测试输入的变化从简单的像素扰动扩展到多层次的变异。这些方法主要服务于深度学习模型测试,重点是让输入样本覆盖更多神经元状态,或者触发不同模型实现之间的行为差异。

在框架级 API 测试方面,Wei 等人提出的 FreeFuzz[10]从开源项目里提取 API 调用模式,再据此构造框架级 API 测试用例。这种方法的特点是利用真实代码里的调用习惯,可以减少人工编写脚本的工作量,也能覆盖较多的框架接口。Deng 等人[11]尝试以大语言模型(Large Language Model, LLM)来生成框架测试用例。他们的实验表明,LLM 能够从语料中学到部分框架 API 签名和参数搭配,也可以产出一部分能够直接运行的测试代码。这些工作为 DL 框架测试输入的自动构造提供了帮助,比较适合处理 API 组合和测试脚本构造这类问题。

不过本文关注的重点和上述工作并不完全一样。已有方法更多回答的是怎么生成测试输入,以及怎么利用代码片段、覆盖反馈或 LLM 来提高脚本生成效率。在测试工具



持续运行之后,系统里还会留下大量测试产物。这些产物如果没有统一管理,后续的相似崩溃归并、低收益用例筛选和补盲方向分析都会比较困难。所以本文更关注生成之后的测试产物管理和分析。具体来说,系统需要把测试产物及其语义向量统一保存下来,再借助历史测试结果完成用例筛选、故障辅助定位以及候选补盲测试脚本的生成。

1.2.2 测试数据管理与效能评估相关研究

测试用例管理领域已在优先级排序和用例约简方向上积累了大量工作。Rothermel 等人 [12] 给出了一种基于覆盖率的贪心排序策略,让覆盖更多新分支的用例优先执行,借此提高回归测试效率。Yoo 等人 [13] 则研究了在满足覆盖率要求的前提下,怎么选出规模更小的测试用例集合。这类方法在传统软件测试中比较常见,一般默认覆盖率向量可以较好地地区分用例差异,测试输入本身也可以用较清晰的结构化特征来描述。在回归测试场景下,这种思路可以减少部分重复执行的成本。

DL 框架模糊测试产物和传统回归测试用例并不完全一样。一方面,测试输入不是简单的标量参数或对象,而是包含计算图结构和参数约束的测试脚本。另一方面,测试结果也不只是通过或失败,还会附带覆盖和运行相关的信息。如果只按文本相似度或覆盖率向量判断用例价值,就容易忽略计算图结构和参数组合上的接近关系。比如两个用例覆盖的代码区域可能相近,但执行成本差别很大;也可能覆盖率差别不明显,但其中一个更接近历史崩溃样本中的参数条件。所以在 DL 框架测试场景下,用例约简和效能评估需要采用更适合高维测试数据的表示方式,并把覆盖贡献和计算成本一起考虑。

近几年向量数据库(Vector Database)技术的发展为非结构化测试产物管理提供了新的工具[14]。Milvus 这类向量检索系统能够在稠密向量上进行近似最近邻(Approximate Nearest Neighbor, ANN)查询,可以在较短时间内完成较大规模的相似度匹配。把语义编码和向量检索引入测试数据管理之后,系统就不再只靠日志里的固定词项或脚本里的字符串片段,也可以按语义接近程度召回历史脚本和崩溃样本。这一点对崩溃日志和计算图脚本这类半结构化数据比较有帮助。

不过,向量检索本身并不能替代覆盖率分析、用例成本评估或候选边界条件判断。它更适合回答“哪些历史样本和当前样本相似”,不能直接说明某个样本是否值得继续执行,也不能单独判断崩溃是不是由某个参数条件触发。因此本文采用 MySQL 和 Milvus 协同的存储方式,把结构化元数据、覆盖率记录和高维语义向量关联起来。在此基础上,系统构建 CEI 评估模型,把覆盖贡献和执行成本结合起来评价;同时结合相似



用例召回和 Daikon 约束分析, 为崩溃原因分析提供参数层面的参考依据。

1.2.3 大语言模型辅助软件测试研究现状

近几年 LLM 在软件工程领域的应用受到较多关注 [15]。在代码生成方面, Codex[16] 和 CodeLlama[17] 等模型已经表现出根据自然语言描述生成程序片段的能力。在缺陷修复方面, 也有研究尝试让 LLM 结合错误上下文生成候选补丁 [18]。这些工作说明 LLM 可以参与测试脚本生成、错误解释和候选修复建议等环节。

不过, 通用 LLM 在处理 DL 框架故障诊断时仍然存在一些限制。框架崩溃日志一般包含较多运行时信息, 直接阅读并不容易。如果模型缺少当前项目里的测试上下文, 就容易按通用经验给出并不适用的解释。这种现象在垂直测试场景下表现为“幻觉”(Hallucination): 比如对框架内部行为给出不准确的解释, 或给出不适用于当前框架版本的参数建议 [19]。所以在故障诊断任务中直接让 LLM 解释堆栈日志, 往往难以保证输出和实际测试环境一致。

检索增强生成 (Retrieval-Augmented Generation, RAG) 架构 [20] 会在模型推理之前先从外部知识库中取出一段相关上下文, 从而对模型输出加以约束。比起直接调用 LLM, RAG 更便于把项目内部的测试上下文交给模型。RAG 通过把相关测试上下文组织成提示词, 让 LLM 不再只凭通用知识解释日志, 而是结合系统里已有的测试数据生成辅助诊断报告和候选补盲测试脚本。

1.3 研究目标与主要工作

本课题围绕 DL 框架模糊测试产物的管理、分析和再利用展开。针对测试过程中产生的各类测试产物, 本文设计了一套面向测试产物管理和故障辅助分析的系统。研究目标不是替代现有的模糊测试生成器, 而是对测试产物做统一管理和分析, 以此提高历史测试数据的复用程度。本文主要从三个方面展开工作。第一项是测试产物的存储和关联。针对测试产物格式不一致的问题, 系统用 MySQL 保存结构化信息, 用 Milvus 管理由脚本和日志编码得到的高维语义向量。这样可以把测试用例、崩溃记录和相似检索结果关联起来, 为后续的缺陷归类和历史样本召回提供数据基础。为此系统完成日志解析、字段抽取、向量生成等流程, 让原本分散在文件系统和日志里的测试产物可以被统一管理和分析。

第二项工作是观察测试集的多样性并量化效能。针对高维测试用例特征, 系统通过



降维映射算法将其投影到低维平面,以便观察测试集在语义空间里出现聚集和留下空缺的位置。同时,系统构建了覆盖效能指数(Coverage Efficiency Index, CEI)评估模型,把单个用例的覆盖贡献和执行成本结合起来,用于识别覆盖收益较低但执行开销较高的冗余用例。和只统计覆盖率不同,CEI 模型关注的是用例在覆盖收益和运行代价之间的关系,为后续的测试集压缩和执行资源分配提供依据。

第三项是崩溃分析。系统把语义检索、约束推断和 RAG 辅助诊断结合起来。系统先通过语义检索召回与目标崩溃用例相似的历史样本,再用基于相似样本对比的 Daikon 候选边界条件分析方法,把语义相似用例召回和参数约束推断结合起来,分析可能触发崩溃的边界条件。在此之上,相似用例上下文、可疑约束和复现信息会被写入 RAG 提示词模板,进而由 LLM 产出辅助诊断报告以及候选补盲测试脚本草稿。

为支撑上述功能,本文首先实现了测试脚本与崩溃日志的解析流程,把日志文本和脚本结构信息编码为语义向量。随后,系统选择 MySQL 承担结构化元数据的保存,用 Milvus 保存向量数据,支持历史用例检索和相似样本召回。在故障分析部分,本文实现了基于深度优先搜索(Depth-First Search, DFS)拓扑回溯的故障辅助定位,以及基于相似样本对比的 Daikon 候选边界条件分析。在用例评估部分,本文实现了基于主成分分析(Principal Component Analysis, PCA)的多样性观察和 CEI 筛选。辅助生成部分则用 RAG 生成辅助诊断报告和候选补盲测试脚本。本文还在插桩编译过的 PyTorch 环境下完成了千例规模的物理对照实验,对 CEI 筛选、边界条件分析和系统性能做验证。



2 相关技术与理论基础

2.1 深度学习框架与模糊测试技术

2.1.1 深度学习框架的体系结构

目前常见的 DL 框架大多采用分层结构 [21]。以 PyTorch 为例，自上而下可以分为三层。其中最上层为面向用户的 Python 接口，向外提供张量操作、自动微分以及神经网络相关模块。中间层负责把用户写出的运算关系组织起来，在执行阶段生成计算图或中间表示 (Intermediate Representation)。为便于后续分析，本文将算子之间的数据依赖抽象为一个有向无环图 (Directed Acyclic Graph, DAG)，节点表示一次算子调用，有向边表示数据的流向。最底层是用 C++ 实现的 ATen 张量计算库和 CUDA 核函数调度层，负责处理 GPU 显存分配、线程调度和底层的数值运算。

这样的三层结构既让上层接口比较易用，也保留了底层的执行效率。但从测试和故障分析的角度来看，分层封装也让用户脚本和底层算子实现之间隔得比较远。当 ATen 库或 CUDA 核函数中某个算子在边界输入下出错时，上层的 Python 堆栈一般只能反映出 API 调用路径，不一定能直接指出真正触发缺陷的底层位置。如果异常还涉及到计算图中前后算子的形状传递或类型转换，只靠日志文本做人工判断会更加困难。本文后续设计的故障辅助定位模块和语义检索流程，就是针对这种跨层封装带来的分析困难设计的。

2.1.2 模糊测试技术原理与分类

Fuzzing 用自动化的方式不断生成测试输入，把这些输入送给待测程序执行，再观察程序运行时是否出现异常信号 [22]。依据输入的构造手法，Fuzzing 通常分成两类。一类是基于变异 (Mutation-based) 的做法，从已有种子输入出发，借助字节翻转、插入或删除得到新的测试用例 [23]；另一类是基于生成 (Generation-based) 的做法，按照输入格式的语法或语义描述，从零构造满足特定约束的测试输入 [24]。

在 DL 框架测试场景下，Fuzzing 输入不再是简单的文件字节流或参数，而是可以直接调用框架 API 的测试脚本。一个用例里通常包含计算图结构和运行配置等信息。因此除了脚本本身，系统还需要把测试执行相关的信息一同记录下来。在 DL 框架测试



中,“崩溃”也不只包括段错误和未捕获异常。差分测试中不同后端给出的数值结果不一致,有时也会提示底层算子的实现可能存在问题[25]。这类问题不一定会产生操作系统层面的异常信号,只看退出状态码也不容易判断。所以系统在处理 Fuzzing 产物时,需要把测试产物和必要的结果信息一起保存下来,并在数据接入阶段完成基本的归档和特征提取。

2.2 文本语义表征与向量检索技术

2.2.1 基于孪生网络的预训练语义编码模型

词袋模型或 TF-IDF 这类做法依靠词项的出现频率得到稀疏向量,能够反映出文本里包含哪些关键词,但在词序、上下文及同义表达的处理上能力有限[26]。这种局限在 DL 框架的崩溃日志中尤为明显:属于同一类底层异常的两条日志,可能因为变量名或调用路径不同而在文本上差别明显;反过来,两条用词相近的日志也不一定来自同一种算子或后端问题。因此本文系统不能只靠关键词匹配,还需要一种能够反映句子语义接近程度的编码方法。

Reimers 等人提出的 Sentence-BERT[27] 在预训练语言模型的基础上引入了孪生网络(Siamese Network)结构,让不同句子得到的嵌入向量可以直接做距离比较。两段文本各自经过共享参数的编码器之后,得到固定维度的稠密浮点向量,再用余弦相似度来衡量它们在语义上的接近程度。这种表示方式比较适合本文系统中的两类数据:一类是崩溃日志和报错堆栈文本,用来从历史记录里找出相似的缺陷;另一类是测试脚本中的计算图结构信息,用来比较不同用例在计算图语义上的接近程度。

本课题采用 all-MiniLM-L6-v2[28] 作为编码器。这个模型由 6 层 Transformer 结构蒸馏而来,输出 384 维向量,在语义区分能力和计算开销之间相对均衡。在本系统里,它放在数据接入流程的向量化环节。每条测试用例入库之前,系统先对日志文本和脚本结构信息分别编码,再把得到的向量标识和 MySQL 中的结构化元数据关联起来保存。这些向量后续用于相似用例的检索和测试集多样性分析。

2.2.2 高维向量近似最近邻检索

当向量的数量不断增多之后,如果每次查询都和库里所有向量逐一计算距离,在线检索的耗时会明显上升。近似最近邻(Approximate Nearest Neighbor, ANN)检索一般会用少量精度损失来换取较快的查询速度[29]。由于本文系统需要在脚本向量和日志向量



库中快速找到与当前用例接近的样本，因而需要一个专门的向量检索系统。

Milvus[30] 是一个面向向量数据的开源检索系统，支持多种索引结构和距离度量。本课题选用 IVF_FLAT (Inverted File Index with Flat quantization) 作为主索引结构。这个索引会先通过聚类把向量空间划分为若干 Voronoi 单元，查询时只在离查询点最近的几个单元里继续比较，把搜索范围从全部样本缩小到局部 [31]。这种方式和本文系统先召回候选相似用例、再结合结构化字段补全信息的查询过程相匹配。

向量之间的距离度量使用余弦相似度。对于两个 d 维向量 $\mathbf{a}$ 和 $\mathbf{b}$ ，余弦相似度的定义如下：

$$\text{sim}(\mathbf{a}, \mathbf{b}) = \frac{\sum_{i=1}^d a_i b_i}{\sqrt{\sum_{i=1}^d a_i^2} \cdot \sqrt{\sum_{i=1}^d b_i^2}} \quad (2.1)$$

其中 a_i 与 b_i 分别为向量 $\mathbf{a}$ 和 $\mathbf{b}$ 的第 i 个分量。余弦相似度主要看两个向量的方向是否接近，受文本长度差异引起的模长变化影响较小。本文系统中的崩溃日志和脚本结构信息长度差别较大，经过 Sentence-BERT 编码之后，用方向上的接近程度来判断语义相似性更合适。所以在相似崩溃召回、相似脚本检索以及 RAG 上下文构造前的候选筛选里，系统都使用这个度量。

2.3 降维分析与不变量推断方法

2.3.1 主成分分析原理

本文系统为每条测试用例生成的语义向量是 384 维。这类向量便于机器检索和相似度计算，但研究人员不容易直接从高维坐标看出测试集的分布。主成分分析 (Principal Component Analysis, PCA) [32] 可以把高维数据投影到低维子空间，同时尽量保留样本方差较大的方向。因此本文用 PCA 生成测试集多样性的可视化结果，观察 Fuzzing 样本是否集中在少数相似的结构附近，或者是否已经覆盖到比较分散的算子组合。

给定 n 个 d 维样本构成的数据矩阵 $\mathbf{X} \in \mathbb{R}^{n \times d}$ (已中心化)，PCA 先计算样本协方差矩阵：

$$\mathbf{C} = \frac{1}{n-1} \mathbf{X}^T \mathbf{X} \quad (2.2)$$

其中 $\mathbf{C} \in \mathbb{R}^{d \times d}$ 是一个对称半正定矩阵。之后再对 $\mathbf{C}$ 做特征值分解：

$$\mathbf{C} \mathbf{v}_k = \lambda_k \mathbf{v}_k, \quad k = 1, 2, \dots, d \quad (2.3)$$



其中 λ_k 是第 k 个特征值, $\mathbf{v}_k$ 是对应的单位特征向量。把特征值按从大到小排好序后, 取前 p 个特征向量组成投影矩阵 $\mathbf{W} = [\mathbf{v}_1, \dots, \mathbf{v}_p] \in \mathbb{R}^{d \times p}$ 。原始数据经过 $\mathbf{Y} = \mathbf{XW}$ 就可以降到 p 维。

本课题把 384 维的测试用例语义向量用 PCA 降到二维平面, 再画出测试集多样性的散点图。如果大量样本在图中集中在某个局部区域, 一般说明当前测试集中结构重复的情况比较多; 如果样本分布相对分散, 就说明测试输入在算子组合或参数结构上的差异更加明显。需要说明的是, PCA 结果只用来辅助观察, 不能直接当作覆盖率指标。在本文系统里, 它主要为后续的 CEI 筛选和补盲方向判断提供参考依据。

2.3.2 Daikon 动态不变量检测框架

Daikon[33] 由 Ernst 等人提出, 用来从程序多次运行的运行时轨迹中推断可能成立的程序不变量 (Program Invariant)。在常见的用法中, Daikon 会在被观测程序的关键位置采集变量的实际取值, 再把这些取值序列和预定义的不变量模板库 (如 $x > 0$ 、 $x = a \cdot y + b$ 、 $x \in \{v_1, v_2, \dots\}$ 等) 做匹配。如果某个模板在所有已观测轨迹上都成立, 就把它作为一条候选不变量输出。

本文沿用 Daikon 中模板匹配和运行时取值分析的思路, 但使用的场景和经典的程序不变量检测不太一样。系统面对的不是单个程序的正常执行轨迹, 而是 DL 框架 Fuzzing 产物里“成功执行”和“崩溃触发”两组对照的数据。对同一类算子或相似的计算图结构, 系统把张量形状和关键参数整理为可比较的数值键值对, 再看哪些约束在健康样本里普遍成立、却在崩溃样本中不再成立。这些约束不能直接当作缺陷的原因, 但可以作为分析崩溃触发条件时的候选边界条件。第四章会进一步说明本文系统怎么把语义相似用例召回和 Daikon 式约束推断结合起来, 形成基于健康组和崩溃组对比的候选边界条件分析方法。

2.4 检索增强生成技术

RAG 架构 [34] 主要由检索器 (Retriever) 和生成器 (Generator) 两部分组成。一般情况下, 检索器先从外部资料中找出和查询相关的片段, 再把这些片段拼到 LLM 的输入提示词里, 生成器据此输出回答。本文系统用到的外部资料不是通用的百科文本, 而是项目内的测试资料和 Daikon 得到的候选约束。

与直接调用 LLM 不同, RAG 更强调把项目内部的上下文喂给模型。借助这种方式,



可以减少模型仅按通用经验作答的情况；后续新增的历史测试数据也能直接补入检索范围，避免因数据更新而重新训练模型权重 [35]。针对 DL 框架故障诊断，本文主要把当前崩溃相关的测试上下文交给 LLM，让 LLM 在已有测试记录的基础上生成辅助的内容。

本课题中 RAG 主要用于两类任务。一是故障辅助诊断任务。系统按当前崩溃用例的语义向量找出历史中的相似案例，再把相关的测试上下文写进提示词，供 LLM 生成辅助诊断报告时参考。二是算子缺口补盲任务。系统把未覆盖或覆盖不足的算子组合写入提示词，再由 LLM 输出候选补盲测试脚本。

2.5 代码覆盖率采集技术

代码覆盖率用来描述测试用例触达了哪些代码区域，也是后续计算用例覆盖贡献的主要数据来源 [36]。GNU 工具链里的 Gcov[37] 是面向 C/C++ 程序的覆盖率采集工具，它的工作过程可以分为三个阶段。在编译阶段，给 GCC 传入 `-coverage` 选项后，编译器会在源代码每个基本块的入口处插入（Instrumentation）计数器指令，同时生成记录源码结构信息的 `.gcno` 文件。在执行阶段，程序运行时各计数器记录实际的命中次数，进程正常退出后再把计数结果写入 `.gcda` 文件。在分析阶段，Gcov 读取 `.gcno` 和 `.gcda` 文件，计算并输出每个源文件的行级和分支级覆盖率。

LCOV[38] 是在 Gcov 输出之上的上层封装工具。它可以对多次运行产生的覆盖率数据做合并（Merge）和差集运算，也能生成 HTML 格式的可视化覆盖率报告。在本课题中，系统需要在插桩编译过的 PyTorch 底层 C++ 源码环境下，逐条执行测试用例并采集对应的覆盖贡献数据。借助 LCOV 的合并和差集功能，系统可以追踪每条用例对全局覆盖率的边际贡献。这部分数据并不单独用来评价测试充分性，而是和算子权重等信息一起输入 CEI 模型，用来判断某条用例和它的执行成本相比是否值得保留。



3 系统需求分析与总体设计

3.1 系统功能需求分析

根据第一章对 DL 框架 Fuzzing 产物管理问题的分析, 本系统围绕测试产物的管理和分析来进行数据处理。进入系统的数据主要来自 Fuzzing 工具生成的测试脚本、执行日志等; 经过解析、向量化和统计分析之后, 系统向开发者提供数据管理界面和分析结果。CEI 评分、故障关联分析结果、辅助诊断报告和候选补盲结果等也会在分析页面中展示。具体的功能性需求如表3.1所示。

表 3.1 系统功能需求分析表

编号	功能需求	输入	输出	补充说明
FR-1	测试数据接入与特征提取	测试脚本、日志文件、算子拓扑结构	结构化元数据、日志向量、结构向量	为检索、统计和后续分析提供基础数据。
FR-2	AI 智能分析助手	自然语言问题、错误描述、历史用例库	相似历史用例、辅助诊断报告	帮助开发者理解当前崩溃与历史样本之间的关系。
FR-3	用例详情故障定位	单条用例日志、错误类型、计算图拓扑	候选算子节点位置、初步分析结果	用于在用例详情中缩小错误位置排查范围。
FR-4	测试集多样性监测	测试用例向量、执行状态、筛选条件	PCA 二维投影结果、分布展示数据	用于观察样本是否集中在少数算子组合附近。
FR-5	测试效能评估与低价值数据清退	覆盖贡献、执行成本、测试用例元数据	CEI 评分、低收益用例标记、清退结果	为冗余用例识别和数据清退提供依据。
FR-6	故障关联分析	召回样例、成功样本、崩溃样本参数	候选边界条件、参数差分、相关样例	用于分析参数组合差异与崩溃现象之间的关联。
FR-7	覆盖分析与算子缺口补盲	覆盖数据、算子组合关系、覆盖缺口	覆盖不足算子衔接、候选补盲测试脚本	将覆盖缺口转化为后续测试输入的候选数据。

3.2 系统总体架构设计

本系统采用分层架构，按照职责划分，系统从下到上分为数据层、算法处理层、服务调度层和应用层四部分，整体结构如图3.1所示。这种划分使界面展示、业务请求、算法计算和数据管理分别由不同层次承担，可以减少不同实现细节之间的耦合。

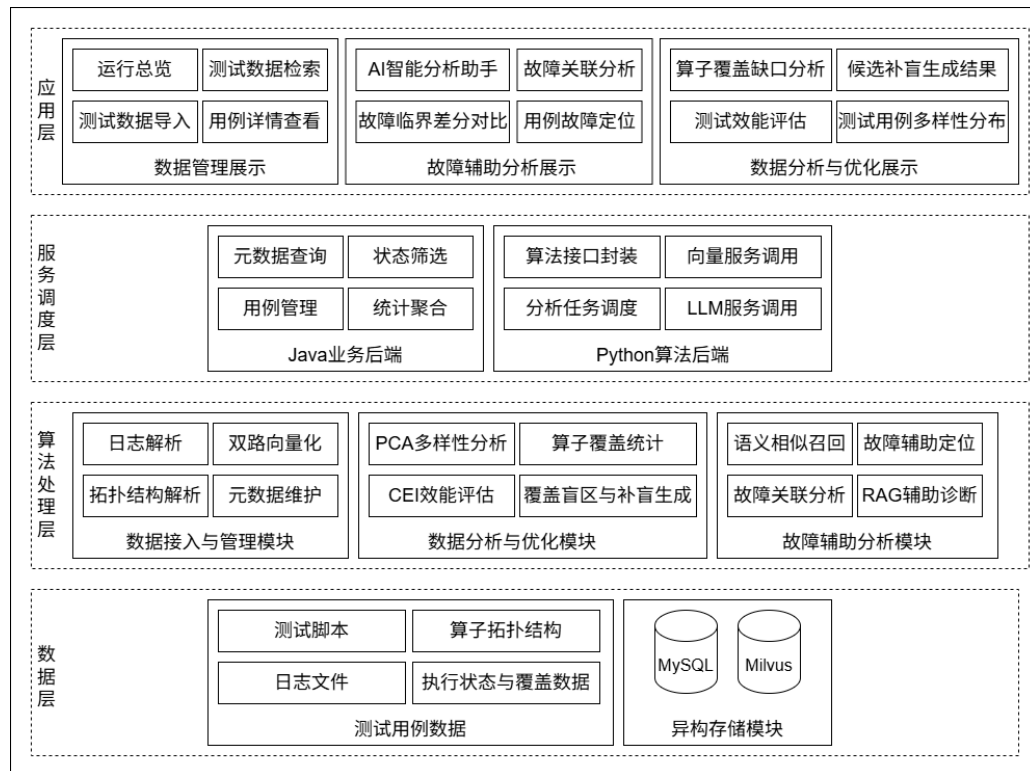


图 3.1 系统总体架构图

数据层保存系统运行所需的数据资源，分为测试用例数据和异构存储模块两部分。测试用例数据由测试脚本、日志文件、算子拓扑结构、执行状态与覆盖数据等组成。异构存储模块由 MySQL 和 Milvus 共同承担，其中 MySQL 保存结构化元数据、状态字段和覆盖统计结果，Milvus 保存日志向量和结构向量，两者共同实现条件查询、相似用例召回，以便后续分析。

算法处理层提供系统分析能力，按照数据处理方向划分为数据接入与管理模块、数据分析与优化模块和故障辅助分析模块。数据接入与管理模块负责日志解析、拓扑结构解析、双路向量化和元数据维护；数据分析与优化模块负责 PCA 多样性分析、算子覆盖统计、CEI 效能评估以及覆盖盲区与补盲生成；故障辅助分析模块负责语义相似召回、故障辅助定位、故障关联分析和 RAG 辅助诊断。这些算法能力根据上层请求被调



用, 处理结果再经由服务调度层返回应用层展示。

服务调度层位于应用层和算法处理层之间, 负责接收前端请求、组织查询或分析流程, 并把结果整理后返回。这一层由 Java 业务后端和 Python 算法后端共同承担。Java 业务后端负责元数据查询、状态筛选、用例管理和统计聚合等业务请求; Python 算法后端负责算法接口封装、向量服务调用、分析任务调度和 LLM 服务调用。具体到实现上, 前端按接口类型选择后端: 常规业务查询、语义检索以及 RAG 问答请求都先走 Java 业务后端, 其中语义检索和 RAG 问答随后由 Java 转发到 Python 算法后端; 数据导入、覆盖分析、效能评估、多样性分析、故障关联分析以及候选补盲生成这类算法请求则直接调用 Python 算法后端。这种划分让结构化业务查询和算法接口调用保持相对清晰的边界。

应用层面向开发者提供 Web 交互界面, 按照展示内容划分为数据管理展示、故障辅助分析展示和数据分析与优化展示三类。数据管理展示包括运行总览、测试数据检索、测试数据导入和用例详情查看, 主要用于管理测试数据和查看详细测试用例信息。故障辅助分析展示包括 AI 智能分析助手、故障关联分析、故障临界差分对比和用例故障定位, 主要用于呈现相似历史用例、候选算子节点、参数差分和辅助诊断报告。数据分析与优化展示包括算子覆盖缺口分析、候选补盲生成结果、测试效能评估和测试用例多样性分布, 主要用于观察测试集覆盖、分布和用例效能。

3.3 系统技术栈选型

基于系统的分层架构, 本系统的技术栈选型围绕前端交互、业务调度、算法处理和数据存储四个方面展开。相关选择主要由系统中的数据形态、算法依赖和展示需求决定。

前端采用 Vue 3 框架配合 TypeScript 开发。本系统前端包含较多状态联动, 包括测试用例筛选条件、分页结果和详情面板等前端状态。Vue 3 的组合式 API 适合把这些逻辑拆成相对独立的状态单元。TypeScript 则用来约束接口返回的数据结构, 减少前后端字段变更带来的运行时错误。UI 组件库选用 Element Plus, 它的表格、表单和对话框等组件能够覆盖管理系统的需求。散点图和热力图渲染基于 ECharts 完成, 以满足管理界面中筛选、悬停和局部查看等交互需求。

业务后端层采用 Java 17 和 Spring Boot 3 框架。这一层面对的是结构化业务数据和前端请求组织, 主要处理常规的业务查询和异常响应。数据访问层使用 Spring Data



JPA/Hibernate 作为 ORM 工具,通过实体类映射 test_case_metadata 表,常规查询由 Repository 接口派生方法完成,统计类的结果则使用 Repository 中的原生 SQL 聚合查询。这一层同时作为部分算法能力的代理入口,主要转发语义检索和 RAG 问答请求。

算法后端层采用 Python 3.11 和 FastAPI 框架。这一层主要依赖语义编码、向量检索和数据处理相关的组件,支撑向量化、检索和数据处理等算法任务。FastAPI 用来把这些算法能力封装成 RESTful 接口,既供 Java 业务后端代理调用,也供前端分析页面直接调用。

存储层采用 MySQL 8.0 和 Milvus 2.3.4 共同管理系统数据。本系统的数据包含两种主要形态:一类是 case_uid、执行状态、覆盖贡献数据和 CEI 得分等结构化字段;另一类是由语义编码模型生成的日志向量和结构向量。结构化字段需要支持条件筛选、分页、排序等,因此由 MySQL 保存。高维语义向量需要支持相似度检索,因此由 Milvus 建立向量索引并完成相似向量召回。每条测试用例入库时,算法后端先把日志向量和结构向量写入 Milvus 并获得向量主键,再把 case_uid、执行状态、覆盖数据和向量主键写入 MySQL。查询时,结构化筛选以 MySQL 为主;语义检索先由 Milvus 返回相似向量主键,再根据 MySQL 中保存的关联字段查找完整元数据。这种异构双库存储方式让 MySQL 负责元数据记录和跨库关联, Milvus 负责向量相似度召回,共同为检索、分析和展示功能提供数据支撑。



4 系统详细设计与主要算法

本章依据第三章给出的系统总体架构图展开详细设计，重点描述算法处理层中的核心模块，以及这些模块与数据层之间的必要关联，包括数据接入与特征表示、异构存储、故障辅助分析以及数据分析与优化等内容。

4.1 数据接入与特征表示设计

数据接入与特征表示设计对应架构图中的数据接入与管理模块。该部分负责把 Fuzzing 产物中的日志、测试脚本和算子拓扑结构整理为可检索、可分析的信息。其中，日志解析用于提取错误类型和运行状态，双路向量化用于把日志语义和模型结构分别映射到向量空间。

4.1.1 基于分层正则匹配的日志解析模块

Fuzzing 工具输出的崩溃日志在格式上不统一。同一条记录中可能同时包含多层运行时信息以及多线程交织产生的乱序输出。日志解析模块接收原始崩溃日志及相关测试信息，将其中稳定的字段提取为后续分析所需的元数据。

本系统设计的 LogParser 模块采用分层正则匹配策略，将解析过程分解为错误分类与字段提取两个独立阶段。在错误分类阶段，模块维护一张带有优先级顺序的错误模式表，涵盖八种 PyTorch 测试中常见的错误类别，如表4.1所示。

表 4.1 LogParser 模块支持的八种标准化错误类别

类别标识	中文描述	典型触发关键词
cuda_oom	CUDA 显存溢出	cuda out of memory, oom
shape_mismatch	张量维度不匹配	size mismatch, incompatible
segfault	内存段错误	segmentation fault, signal 11
timeout	执行超时	timeout, exceeded maximum
device_mismatch	设备不匹配	expected.*device
index_error	索引越界	index out of, out of bounds
type_error	类型错误	type error, invalid type
nan_inf	数值异常	nan, inf, not a number



模块将输入文本统一转换为小写后，依照表中自上而下的优先级顺序扫描关键词。一旦命中即终止扫描并返回对应错误类别。优先级排列主要依据异常处理时的排查优先级：显存溢出与段错误属于系统级致命异常，排在最前；数值异常属于静默型偏差，排在最后。对于不命中任何模式的日志，模块将其归入 other 类别。该分类结果用于为后续故障辅助定位和检索分析提供错误标签。

字段提取阶段则针对不同数据类型分别准备了一组正则表达式，从原始日志中抽出以下六项结构化字段：

(1) 错误类型 (error_type)。面向 Python 异常，模块使用形如 $(\backslash w+Error):$ 的正则匹配得到类名；遇到 C++ 层的段错误，则改为检测是否出现 Segmentation Fault 关键字。

(2) 张量维度 (tensor_shapes)。PyTorch 的维度报错主要包含方阵表示法 $m1: [32 \times 512]$ ，部分报错还会直接给出匿名形状 $[32 \times 512]$ 。模块为这两类格式分别编写正则表达式并按优先级依次尝试，成功匹配后输出为 {name, shape} 结构化字典列表。

(3) 框架源码位置 (source_location)。C++ 侧报错一般带有类似 at /pytorch/.../THTensorMath.cpp:41 的源码路径，Python 侧则使用 File "xxx.py", line 42 这样的格式。模块分别准备了正则表达式，用于取出文件名、行号和模块路径。

(4) 显存信息 (memory_info)。仅在 cuda_oom 类别下激活。模块提取三组数值：请求分配量、已分配量与设备总容量。

(5) 设备信息 (device_info)。通过全局扫描 cuda:N 与 cpu 标识符提取，去重后输出为列表。

(6) 原始消息 (raw_message)。保留未经处理的完整日志文本，供 RAG 模块作为 LLM 上下文素材。

4.1.2 双路向量化特征表示方法

如第二章所述，本系统采用 Sentence-BERT 编码模型将离散文本映射为 384 维稠密浮点向量。针对 Fuzzing 产物的异构特性，系统设计了两条并行的向量化路径。

第一条路径为报错堆栈语义表征。模块把用例的 error_message 字段直接送入 Sentence-BERT 编码器。遇到成功执行、没有错误信息的使用例，则补充占位文本 "Success: [status] - Model executed OK" 进行编码。这条路径产出的向量写入 Milvus 中的 fuzzing_logs 集合，供后续语义检索使用。



第二条路径为图拓扑序列投影。模块将测试用例的模型结构字典序列化为标准化文本, 格式为"Model structure: layers=[...], connections=..., depth=N, operators=[...]"。该文本经 Sentence-BERT 编码后产出的向量存入 Milvus 的 fuzzing_structure 集合, 用于多样性分析与拓扑相似性召回。

为了避免重复加载开销, 两条路径共用同一个 all-MiniLM-L6-v2 模型实例。每个测试用例经过该处理后会得到两个独立的 384 维向量。

4.2 异构存储模块设计

异构存储模块用于连接数据接入的结果与后续分析算法。结合第三章的技术栈选型, 系统采用 MySQL 与 Milvus 组合保存测试用例数据。

在数据组织上, MySQL 中的 test_case_metadata 表以 case_uid 作为业务标识, 保存用例状态、错误信息、分支边覆盖数、执行耗时和扩展参数等信息。Milvus 维护 fuzzing_logs 和 fuzzing_structure 两个集合, 分别对应报错堆栈语义向量和模型结构向量。两类数据不放在同一数据库中, 主要是因为结构化筛选和相似度检索对存储模型的要求不同。

跨库关联通过 MySQL 中保存的向量主键完成。数据接入阶段, 系统先完成日志文本和结构文本的向量化, 再将两路向量写入 Milvus 并取得各自的主键标识 (structure_vector_id 与 log_vector_id)。结构向量主键写入 MySQL 的 vector_id 字段, 结构向量主键与日志向量主键同时写入 parameters 字段, 并与结构化元数据和 case_uid 一同保存。后续检索时, 系统可先在 Milvus 中召回相似向量, 再依据返回的主键查询 MySQL 中的元数据。这种设计避免在 Milvus 中保存业务字段, 也便于在 MySQL 中维护用例状态、覆盖数据和清退标记。

4.3 故障辅助分析模块设计

故障辅助分析模块对应系统架构图中的故障辅助分析能力, 主要包括单用例故障辅助定位、语义相似召回、故障关联分析和 RAG 辅助诊断。其中, 故障辅助定位用于在用例详情中标注候选算子节点; 语义相似召回和 RAG 辅助诊断用于组织历史案例并生成辅助诊断报告; 故障关联分析则结合相似成功样本与崩溃样本, 提取候选边界条件和参数差分。

4.3.1 基于 DFS 拓扑回溯的故障辅助定位模块

日志解析完成后, 系统需要将结构化错误信息与测试用例的计算图结构相结合, 定位出可能与崩溃相关的候选算子节点位置。本系统设计的 FaultLocator 模块通过逆向邻接表与 DFS 回溯实现该功能, 帮助用户缩小排查范围。

逆向邻接表的构建过程如下。系统将测试用例中的算子拓扑统一表示为有向图 $G = (V, \mathcal{E})$, 其中节点集合 V 表示算子节点, 边集合 $\mathcal{E}$ 表示数据的流向。设正向边 $(o_i, o_j) \in \mathcal{E}$ 表示数据由 o_i 流向 o_j , 则逆向邻接表中记录 o_j 到 o_i 的反向关系。

具体定位过程如算法1所示。

Algorithm 1 基于 DFS 回溯的候选算子定位算法

Require: 结构化错误信息 E , 算子有向图 $G = (V, \mathcal{E})$

Ensure: 候选算子节点列表 $\mathcal{L}$

- 1: 根据边集合 $\mathcal{E}$ 构建逆向邻接表 $\mathbf{A}^{-1}$
 - 2: 根据 E 中的错误类别选择错误相关算子集合 $\mathcal{M}$
 - 3: 根据 E 确定候选触发节点集合 $\mathcal{T}$; 若无法确定, 则令 $\mathcal{T} \leftarrow V$
 - 4: $\mathcal{L} \leftarrow \emptyset$
 - 5: for each trigger node $t \in \mathcal{T}$ do
 - 6: 从 t 开始沿 $\mathbf{A}^{-1}$ 执行 DFS 回溯
 - 7: for each visited node o_i do
 - 8: if $o_i \in \mathcal{M}$ or o_i matches a risky topology pattern then
 - 9: 计算候选置信度 $score(o_i)$ 并生成推理理由
 - 10: $\mathcal{L} \leftarrow \mathcal{L} \cup \{(o_i, id(o_i), score(o_i), reason)\}$
 - 11: end if
 - 12: if $o_i \in \mathcal{M}$ or depth exceeds threshold or $\mathbf{A}^{-1}(o_i) = \emptyset$ then
 - 13: 停止当前回溯分支
 - 14: end if
 - 15: end for
 - 16: end for
 - 17: 对 $\mathcal{L}$ 按置信度降序排序
 - 18: return $\mathcal{L}$
-

其中, 错误相关算子集合 $\mathcal{M}$ 根据错误类型确定。例如, 维度变换错误主要关注 Linear、Conv2d 和 Reshape 等算子; 高显存问题优先检查 Conv 类和 Attention 类算子。这里的置信度只用于排序候选节点, 不表示真实缺陷概率。该列表将传递给前端在计算图可视化中高亮标注候选节点。

4.3.2 语义相似召回与 RAG 辅助诊断

当用户针对某一崩溃用例发起诊断请求时，系统先从历史用例中召回语义相近的记录，再把召回结果作为 RAG 上下文交给 LLM 生成辅助诊断报告。其处理过程如算法2所示。

Algorithm 2 语义相似召回与 RAG 辅助诊断算法

Require: 用户查询 q ，最终返回数量 K

Ensure: 召回用例集合 C ，辅助诊断报告 R

- 1: $v_q \leftarrow \text{SentenceBERT}(q)$
 - 2: 在 `fuzzing_structure` 中检索与 v_q 最相似的前 $3K$ 个结构候选
 - 3: 在 `fuzzing_logs` 中检索与 v_q 最相似的前 $3K$ 个日志候选
 - 4: 根据 q 中的结构关键词和日志关键词比例计算两路融合权重
 - 5: 合并两路候选结果，并按融合得分截取前 K 个用例，记为 C
 - 6: 根据 C 中的主键从 MySQL 反查完整元数据
 - 7: 将元数据和用户查询 q 组装为 RAG 上下文提示词 P
 - 8: $R \leftarrow \text{LLM}(P)$
 - 9: return C, R
-

LLM 基于检索到的真实历史案例生成辅助诊断报告，为开发者提供故障排查参考信息。

4.3.3 故障关联分析

故障关联分析中的候选边界条件分析模块借鉴经典的 **Daikon** 动态不变量推断范式 [33]，在成功样本与崩溃样本构成的对照组中扫描候选约束，输出候选边界条件，用于辅助分析崩溃触发条件。

候选边界条件分析的输入为一个目标崩溃用例。模块先提取该用例的图拓扑结构向量，在 Milvus 的 `fuzzing_structure` 集合中执行余弦相似度检索，分别召回拓扑结构相似但执行成功的 10 个用例（健康组 $\mathcal{S}$ ）与执行失败的 10 个用例（崩溃组 $\mathcal{F}$ ）。召回条件为结构相似度大于 0.85，确保对照样本与目标用例的算子组合足够相近。

召回完成后，模块对每个用例的嵌套参数字典执行展平操作。DL 测试用例的参数空间通常包含模型结构字典、层配置子字典、标量参数和列表参数。展平操作把这些嵌套结构转成一维键值对，同时过滤向量主键、覆盖率等无关字段。过滤后的键值对集合仅保留与算子配置直接相关的参数。

展平完成后，模块计算健康组与崩溃组所有样本的键集合交集，其中 $\Phi(\cdot)$ 表示参数

展平后的键值对集合:

$$\mathcal{K} = \bigcap_{s \in \mathcal{S}} \text{keys}(\Phi(s)) \cap \bigcap_{f \in \mathcal{F}} \text{keys}(\Phi(f)) \quad (4.1)$$

仅在 $\mathcal{K}$ 中的键才进入后续的候选约束扫描环节。

模块在公共键集合 $\mathcal{K}$ 上扫描三层模板池, 包括单变量阈值与奇偶性模板、双变量约束与整除关系模板, 以及三变量约束模板。仅当某条候选约束同时满足“成功全票通过”与“失败全票否决”的双重验证时, 该约束才被加入候选边界条件集合。

上述三层扫描的双重验证判据以算法3的伪代码形式给出。

Algorithm 3 双重验证判据——候选边界条件提取算法

Require: 健康组展平参数集合 $\mathcal{S}_{\text{flat}} = \{s_1, s_2, \dots, s_{|\mathcal{S}|}\}$
Require: 崩溃组展平参数集合 $\mathcal{F}_{\text{flat}} = \{f_1, f_2, \dots, f_{|\mathcal{F}|}\}$
Require: 公共键集合 $\mathcal{K} = \{k_1, k_2, \dots, k_n\}$
Ensure: 候选约束集合 $\mathcal{R} = \{\}$

```

1: for each candidate predicate  $\phi(k_{i_1}, \dots, k_{i_p})$  in template pool do
2:   success_pass  $\leftarrow$  True
3:   failure_reject  $\leftarrow$  True
4:   for each  $s \in \mathcal{S}_{\text{flat}}$  do
5:     if  $\phi(s[k_{i_1}], \dots, s[k_{i_p}])$  is False then
6:       success_pass  $\leftarrow$  False
7:       break
8:     end if
9:   end for
10:  if success_pass is True then
11:    for each  $f \in \mathcal{F}_{\text{flat}}$  do
12:      if  $\phi(f[k_{i_1}], \dots, f[k_{i_p}])$  is True then
13:        failure_reject  $\leftarrow$  False
14:        break
15:      end if
16:    end for
17:  end if
18:  if success_pass is True and failure_reject is True then
19:     $\mathcal{R} \leftarrow \mathcal{R} \cup \{\phi\}$ 
20:  end if
21: end for
22: return  $\mathcal{R}$ 

```

提取出的约束集合 $\mathcal{R}$ 经去重后按优先级排序: 多变量约束排在单变量约束之前, 因为多变量约束包含更丰富的变量间交互信息。排序后的约束列表即为该崩溃用例的候选



边界条件集合，用于为开发者提供后续排查线索。

最后，系统把候选约束集合 $\mathcal{R}$ 、参数差分和相关测试上下文组装为提示词，由 LLM 生成用于解释候选约束的辅助诊断报告。前端将候选数学公式和辅助诊断报告并列展示。

4.4 数据分析与优化模块设计

数据分析与优化模块对应架构图中的数据分析与优化能力。该模块针对测试集整体质量，提供多样性分布观察、效能评估、算子覆盖缺口分析以及候选补盲测试脚本生成等功能。

4.4.1 基于 PCA 的测试集多样性分析

Fuzzing 过程会不断产生测试用例。这些用例在语义向量空间中的分布，可以用于观察测试集在算子组合和参数结构上的差异。由于 384 维向量空间不便直接查看，系统需要先将它压缩到二维平面。

本系统在批量更新后或用户请求多样性分析时，对当前样本集合统一执行 PCA 降维。具体流程为：取当前参与分析的测试用例结构向量矩阵 $\mathbf{X} \in \mathbb{R}^{b \times 384}$ (b 为参与降维的样本数量)，计算协方差矩阵并提取前两个主成分方向，将每个样本投影至二维坐标 (x_i, y_i) 。投影后的坐标经最小-最大归一化映射至 $[-1, 1]$ 区间，存入 MySQL 的 parameters 字段中的 vis_x 与 vis_y 键。

前端散点图基于上述二维坐标渲染。用户可观察测试集的空间分布形态：若散点呈现明显的簇状聚集，说明当前 Fuzzing 策略可能集中在有限的算子组合区域内；若散点分布较为分散，说明测试输入在模型结构或参数组合上具有更高差异。稀疏区域对应尚未被充分测试的算子拓扑组合，可作为后续补盲策略的参考方向。

4.4.2 CEI 覆盖效能指数量化模型

CEI 用来描述单个测试用例在覆盖贡献数据和计算成本之间的关系，其数学定义如下：

$$\text{CEI} = \frac{E_{\text{edge}}}{\sum_{i=1}^m w_i \cdot c_i} \quad (4.2)$$



其中 E_{edge} 为该用例的分支边覆盖数, m 为该用例计算图中不同算子类型的数量, c_i 为第 i 类算子在计算图中的出现次数, w_i 为第 i 类算子对应的权重系数。公式分母表示按算子类型和出现次数估计的计算成本, 不包含执行耗时变量; 执行耗时可以作为实验评估指标单独记录。

算子权重 w_i 的设计原则是: 计算量越大、参数空间越大的算子, 赋予的权重越大, 因为这类算子消耗的执行资源更多, 却不一定带来更高的覆盖贡献。表4.2列出本系统的部分算子权重。

表 4.2 部分算子的 CEI 权重基准

算子类别	典型算子	权重 w_i
计算密集型	Conv2d, Conv3d, Linear, LSTM	3.0
注意力机制	MultiHeadAttention, Transformer	4.0
归一化层	BatchNorm2d, LayerNorm, GroupNorm	1.5
池化层	MaxPool2d, AvgPool2d, AdaptiveAvgPool	1.0
激活函数	ReLU, Sigmoid, Tanh, GELU	0.5
结构操作	Flatten, Reshape, Dropout, Concat	0.3

系统在执行清退时, 把全库 CEI 中位数的 α 倍作为淘汰阈值。CEI 低于这个阈值的用例在当前指标下覆盖收益相对较低, 会进入级联删除流程。这一流程主要帮助识别冗余数据, 减少它们对存储空间和后续分析结果的干扰。

当用户确认执行清退操作时, 系统需要按照 MySQL 中保存的向量主键同步处理 Milvus 向量记录。具体流程分为三步。第一步, 从 MySQL 中批量查询满足清退条件的记录, 提取其 `vector_id`、`parameters.structure_vector_id` 和 `parameters.log_vector_id`。第二步, 调用 Milvus 的 `delete` 接口按主键删除两个集合中的对应向量。第三步, 在 MySQL 中执行 `DELETE` 语句移除对应元数据记录。

4.4.3 算子覆盖缺口分析

除了对单个用例做效能评估之外, 系统还需要从全局视角分析测试盲区, 即在当前已出现算子和常用算子范围内, 尚未被测试集覆盖的算子组合模式。

系统不直接枚举 PyTorch 中的全部公开算子, 而是构造用于缺口分析的候选算子集合 O 。该集合由两部分组成: 一部分是当前测试集中已经出现过的算子类型, 另一部分



是预设的 20 个常用 PyTorch 算子类型。记当前测试集已出现算子集合为 O_{seen} ，常用算子集合为 O_{common} ，则有：

$$O = O_{\text{seen}} \cup O_{\text{common}} \quad (4.3)$$

全局邻接矩阵 $\mathbf{A} \in \{0, 1\}^{|O| \times |O|}$ 记录了当前测试集中已经出现过的算子对组合关系。

盲区提取通过集合差运算来完成。定义候选可行算子对全集 $\mathcal{E}_{\text{all}}$ 为 O 中所有在 PyTorch 算子语义下合法的算子组合（排除不合理的组合，例如 Softmax 不能直接接 Conv2d 的通道输入端）。已覆盖算子对集合 $\mathcal{E}_{\text{covered}} = \{(i, j) \mid \mathbf{A}[i][j] = 1\}$ 。盲区集合定义为：

$$\mathcal{B} = \mathcal{E}_{\text{all}} \setminus \mathcal{E}_{\text{covered}} \quad (4.4)$$

盲区集合 $\mathcal{B}$ 中的每个元素 (o_i, o_j) 表示一对还没被测试的算子衔接。系统把 $\mathcal{B}$ 按算子类别分组后输出为结构化列表。这一列表会作为后续补盲模块的输入目标。

4.4.4 覆盖缺口驱动的候选补盲测试脚本生成

盲区集合 $\mathcal{B}$ 标记了当前测试集的覆盖缺口。本系统通过构造结构化提示词，让 LLM 生成候选补盲测试脚本来覆盖这些缺口。

结构化提示词的设计如表4.3所示：

上述四板块模板在工程实现中的作用如下：**System** 板块限定 LLM 的角色边界，减少它偏离测试生成任务；**Context** 板块注入真实的历史数据作为参考，减少无上下文生成的问题；**Target** 板块指定覆盖目标，减少 LLM 生成已有冗余用例的可能；**Constraint** 板块给出语法和结构约束，让输出更便于后续检查和执行验证。

LLM 生成的候选补盲测试脚本经过必要的检查和确认后，可以作为后续测试输入的候选材料，再由开发者确认是否进入数据接入流程，形成“发现盲区 → 生成候选补盲测试脚本 → 执行验证 → 后续分析”的处理流程。



表 4.3 RAG 补盲提示词模板结构

板块	内容设计
System	“你是一个深度学习模糊测试专家助手。你的任务是根据用户问题和检索到的测试用例上下文，提供专业的分析和建议。请基于提供的测试用例数据给出具体建议；如果问题涉及错误分析，给出可疑条件和排查方向；如果问题涉及优化建议，给出可行的改进方案。保持回答简洁专业，使用中文回答。”
Context	注入从 Milvus 召回的 K 个最相似历史用例的结构化摘要，每条包含：用例编号、执行状态和错误信息摘要等。格式为“用例 N: [case_uid] - 状态: [status] - 算子: [operators]”。
Target	明确列出当前需要补盲的算子对列表，例如“请生成一个包含 Conv2d->GroupNorm->LSTM连接链的 PyTorch 模型脚本”。包含目标算子的参数范围约束（如 in_channels $\in$ [1,512]、kernel_size $\in$ [1,7]）。
Constraint	语法限制与输出格式要求：“输出应为可检查的候选 Python 脚本；模型应继承 torch.nn.Module；forward 方法应接受形状为 [batch, channels, H, W] 的输入张量；脚本末尾应包含推理调用语句。”



5 系统实现与功能展示

5.1 开发环境与工具配置

本系统的开发、测试和部署涉及前端、业务后端、算法后端和底层物理执行四个层面。为了让系统实现和后续实验环境保持一致，本章和第六章采用同一套软硬件配置。表5.1列出系统实现和实验验证所使用的主要环境配置。

表 5.1 开发环境与工具配置清单

类别	名称	版本/规格	用途
操作系统	Windows 11 + WSL2 (Ubuntu 22.04)	23H2	开发与物理执行
处理器	Intel Core i7-12700H	14 核 20 线程	编译与实验运行
内存	DDR4	16 GB	数据导入与服务运行
显卡	NVIDIA RTX 3060 Laptop GPU	6 GB GDDR6	CUDA 算子执行
CUDA 工具链	CUDA Toolkit	11.8	GPU 运行时环境
前端框架	Vue 3 + TypeScript	3.3 / 5.3	单页应用开发
UI 组件库	Element Plus	2.4	界面组件渲染
构建工具	Vite	5.0	前端热更新打包
业务后端	Java Spring Boot + JPA/Hibernate	3.2 / 8080 端口	结构化数据管理与代理转发
JDK	OpenJDK	17	Java 运行时
算法后端	Python FastAPI	0.104 / 5000 端口	算法服务与 RAG
Python	CPython	3.11	算法后端运行时
关系数据库	MySQL	8.0	结构化元数据存储
向量数据库	Milvus	2.3.x	高维语义向量存储
语义编码	all-MiniLM-L6-v2	384 维输出	日志与脚本结构向量化
深度学习框架	PyTorch	2.1.0 源码编译	物理执行与覆盖率采集
覆盖率工具	Gcov / LCOV	GCC 11 / 1.16	C++ 层探针采集
前端图表	ECharts	—	散点图、热力图与统计图表展示



当前实现中，前端运行在 Vite 开发服务器上（默认端口 5173），通过 Axios 访问 Java Spring Boot 后端和 Python FastAPI 后端。Java 后端（端口 8080）主要负责结构化数据管理与部分代理转发，Python 后端（端口 5000）主要负责语义检索、候选边界条件分析、CEI 分析和候选补盲建议等算法处理。

底层物理执行环境部署在 WSL2 中的 Ubuntu 22.04 子系统里。这个子系统里安装了带 Gcov 探针的 PyTorch 编译版本，以及独立的 Python 虚拟环境 venv_pytorch。物理执行层通过 multiprocessing 模块的 spawn 启动方式创建独立的子进程，在 WSL 环境下调用插桩后的 PyTorch 库执行测试脚本，采集覆盖贡献数据。

5.2 后端主要模块实现

5.2.1 Python FastAPI 算法后端实现

Python FastAPI 算法后端主要负责语义编码、向量检索、PCA 降维、Daikon 候选边界条件分析和 RAG 相关接口。服务初始化阶段建立 Milvus 连接、加载两个向量集合及 Sentence-BERT 编码模型；MySQL 连接则在处理具体请求时按需建立。

该后端注册了数据导入、语义检索、统计分析、候选边界条件分析、CEI 清退和补盲建议等 API 路由。表5.2列出主要路由的方法、路径和功能摘要。

表 5.2 Python FastAPI 算法后端 API 路由清单

方法	路径	功能摘要
POST	/api/ingest	批量导入测试用例，执行双路向量化与写入
POST	/api/search	语义检索并返回匹配用例
POST	/api/ask	RAG 辅助诊断问答
GET	/api/recent_cases	获取最近入库的测试用例列表
GET	/api/statistics	全局统计概览
GET	/api/analysis/coverage-gaps	算子覆盖热力图与覆盖缺口统计
GET	/api/analysis/efficiency	CEI 效能统计与散点分布
GET	/api/analysis/error-patterns	错误模式交叉聚类分析
GET	/api/analysis/correlations	算子-错误关联矩阵
POST	/api/analysis/fault-boundary	候选边界条件分析与辅助诊断报告生成
POST	/api/suggest	基于历史数据的变异建议生成
POST	/api/auto-generate-from-gaps	覆盖盲区候选补盲测试脚本生成
POST	/api/analysis/efficiency/prune	CEI 阈值清退预览与执行



语义检索接口/api/search 的请求体以 query 字段为核心,用于接收自然语言查询文本。请求模型还包含 top_k、search_mode、structure_weight 和 log_weight 等字段,用于描述检索模式和结构向量与日志向量的融合权重;响应体包含 total、results、query、search_mode 和 query_intent 等字段,其中 results 数组保存匹配用例及其相似度得分。

候选边界条件分析接口/api/analysis/fault-boundary 的请求体包含 failed_case_uid 字段,用于指定待分析的崩溃用例。响应体包含 failed_case、successful_case、diff、batch_context、daikon_violations、daikon_validation 和 boundary_rule 等字段。其中,diff 用于展示参数差异,daikon_validation 保存待人工验证的正反样例,boundary_rule 保存辅助诊断报告。

5.2.2 Java Spring Boot 业务后端实现

Java Spring Boot 后端承担基础业务数据管理和前端请求组织功能,监听 8080 端口,并通过 Spring Data JPA 与 MySQL 交互。该后端提供的主要 API 路由如表5.3所示。

表 5.3 Java Spring Boot 业务后端 API 路由清单

方法	路径	功能摘要
GET	/api/cases	分页查询测试用例,可按状态过滤
GET	/api/cases/recent	获取最近测试用例,用于运行总览展示
GET	/api/cases/{id}	按物理主键查询单个测试用例
GET	/api/cases/uid/{caseUid}	按业务标识符查询单个测试用例
GET	/api/statistics/overview	返回总数、崩溃数、平均耗时和平均覆盖数据
GET	/api/statistics/status	返回不同状态的用例数量
GET	/api/statistics/errors	返回错误类型排行
POST	/api/search	代理语义检索请求到 Python 服务
POST	/api/ask	代理 RAG 辅助诊断请求到 Python 服务

其中,统计接口组包含三条路由:/api/statistics/overview 返回用例总数、崩溃率和平均执行耗时;/api/statistics/status 返回各状态类别的用例数;/api/statistics/errors 返回错误类型。这些结果会被封装为前端图表组件可以直接使用的 JSON 格式。

对语义检索和 RAG 辅助诊断请求,Java 后端通过 Spring Cloud OpenFeign 转发到配置项 python-service.url 对应的 Python 服务,再把响应体返回给前端。



5.2.3 数据库表结构与向量集合详细设计

表 test_case_metadata 负责存储测试用例结构化元数据。表5.4列出了该表的完整字段设计。

表 5.4 MySQL 主要表 test_case_metadata 字段设计

字段名	数据类型	主键	说明
id	BIGINT	是	自增物理主键
case_uid	VARCHAR(64)	唯一	全局唯一业务标识符
status	VARCHAR(20)	—	执行状态 (Success/Crash/Timeout)
edge_coverage	INT	—	Gcov 采集的分支边覆盖数
execution_time	FLOAT	—	执行耗时 (秒)
error_message	TEXT	—	原始崩溃日志文本
vector_id	BIGINT	—	Milvus 结构向量集合中的主键
parameters	JSON	—	扩展信息 (模型结构、双向量 ID、PCA 坐标等)
created_at	DATETIME	—	记录创建时间戳
updated_at	DATETIME	—	记录最后更新时间戳

在 case_uid 字段上建立唯一索引, 通过 ON DUPLICATE KEY UPDATE 语义在重复导入时更新已有记录。parameters 字段采用 JSON 类型, 保存模型结构字典、PCA 降维坐标和 Milvus 双向量主键等扩展信息。其中 Milvus 双向量主键包括 structure_vector_id 和 log_vector_id。

Milvus 侧维护两个独立的向量集合, Schema 配置相同。表5.5列出了集合的字段与索引参数。

两个集合在服务启动时执行 collection.load() 操作, 把索引结构和向量数据加载到 Milvus 的查询节点内存中, 从而降低检索请求触发额外加载的开销。

跨库数据关联通过写入顺序和主键回填共同维护: 结构向量主键写入 vector_id 字段, structure_vector_id 和 log_vector_id 写入 parameters JSON 字段。清退时, 系统先删除 Milvus 中的对应向量, 再删除 MySQL 中的元数据记录。



表 5.5 Milvus 向量集合 Schema 与索引配置

配置项	参数	值
集合名称（结构向量）	—	fuzzing_structure
集合名称（日志向量）	—	fuzzing_logs
主键字段	id	INT64, 自动生成
向量字段	embedding	FLOAT_VECTOR, 维度 384
索引类型	index_type	IVF_FLAT
相似度度量	metric_type	COSINE
倒列表数量	nlist	128
检索探测簇数	nprobe	16

5.3 前端界面实现与功能展示

本系统前端采用 Vue 3 和 TypeScript 实现，界面组件主要基于 Element Plus 搭建。页面整体以深色风格为主，通过卡片、表格、弹窗等形式展示测试用例、统计结果和分析信息。在实现上，前端按页面功能发起 HTTP 请求，其中用例管理和基础统计主要访问 Java 后端，数据导入、故障分析和补盲建议等功能则调用 Python 后端接口。

5.3.1 FuzzHub 运行总览

FuzzHub 运行总览（Dashboard）是用户进入系统后的首屏，主要用于查看测试任务的整体情况。页面上方用统计卡片展示累计用例数、通过率、崩溃总数和平均执行耗时，中部展示测试状态分布，底部列出最近产生的测试用例。这些数据主要由后端统计接口和最近用例查询接口提供。通过该页面，用户可以先对当前测试集的规模、失败比例和近期执行情况有一个整体认识。

5.3.2 用例管理界面

用例管理页面（Cases）用于浏览和筛选测试用例。用户可以按照执行状态、用例编号等条件查看记录，也可以进入单条用例的详情页面。表格中主要展示用例 ID、执行状态、覆盖数据、运行耗时和错误摘要等信息。该页面承担了测试数据入口的作用，便于用户从大量用例中快速找到需要进一步分析的失败样例。测试用例管理界面如图5.2所示。

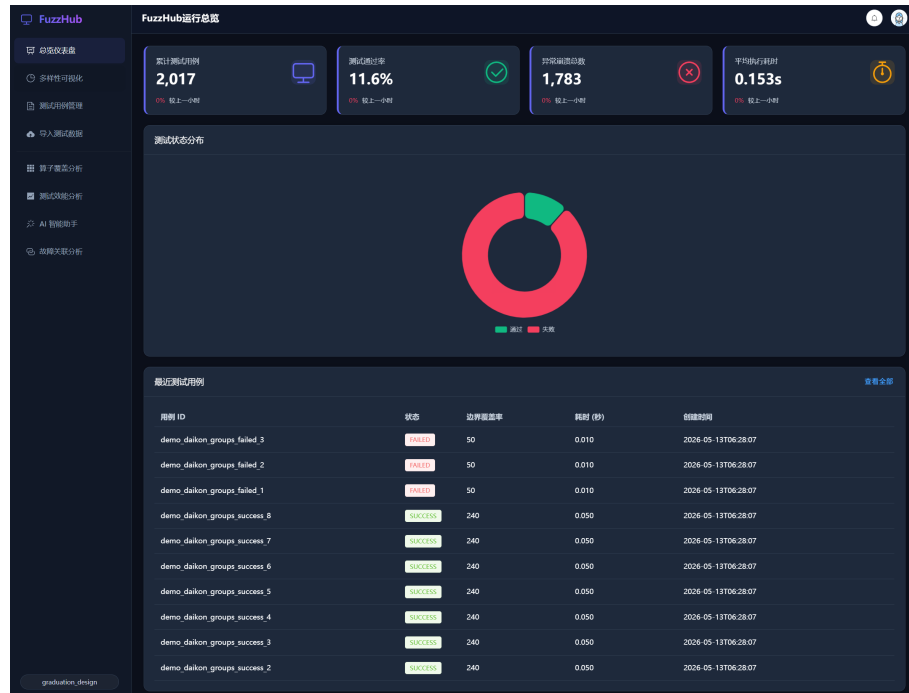


图 5.1 FuzzHub 运行总览界面

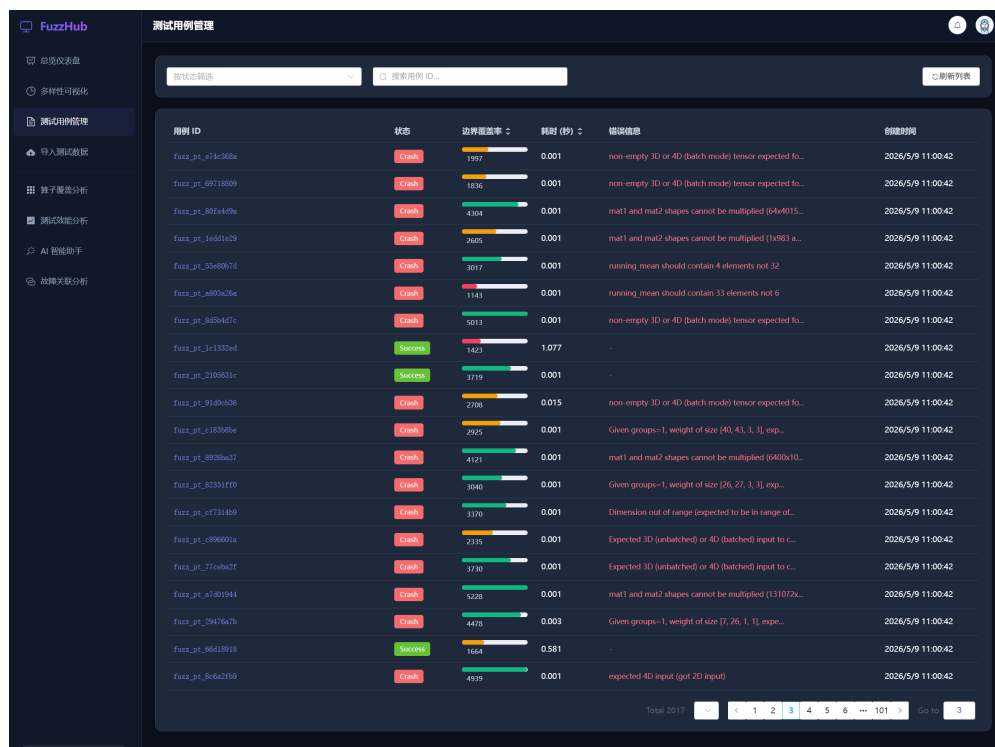


图 5.2 测试用例管理界面

5.3.3 用例详情故障辅助定位界面

用例详情界面用于展示单条测试用例的完整信息。除用例 ID、执行状态、运行耗时和覆盖数据外，页面还会展示错误日志、模型结构和输入参数等内容。对于失败用例，前端会调用 Python 后端分析接口，给出错误类别、候选算子节点和相关说明，帮助用户缩小排查范围。

该界面还提供候选边界条件分析入口。用户可以针对某个失败用例发起候选边界条件分析，查看成功用例与失败用例之间的参数差异，并生成满足或不满足候选约束的待人工验证样例。用例详情故障辅助定位界面如图5.3所示。

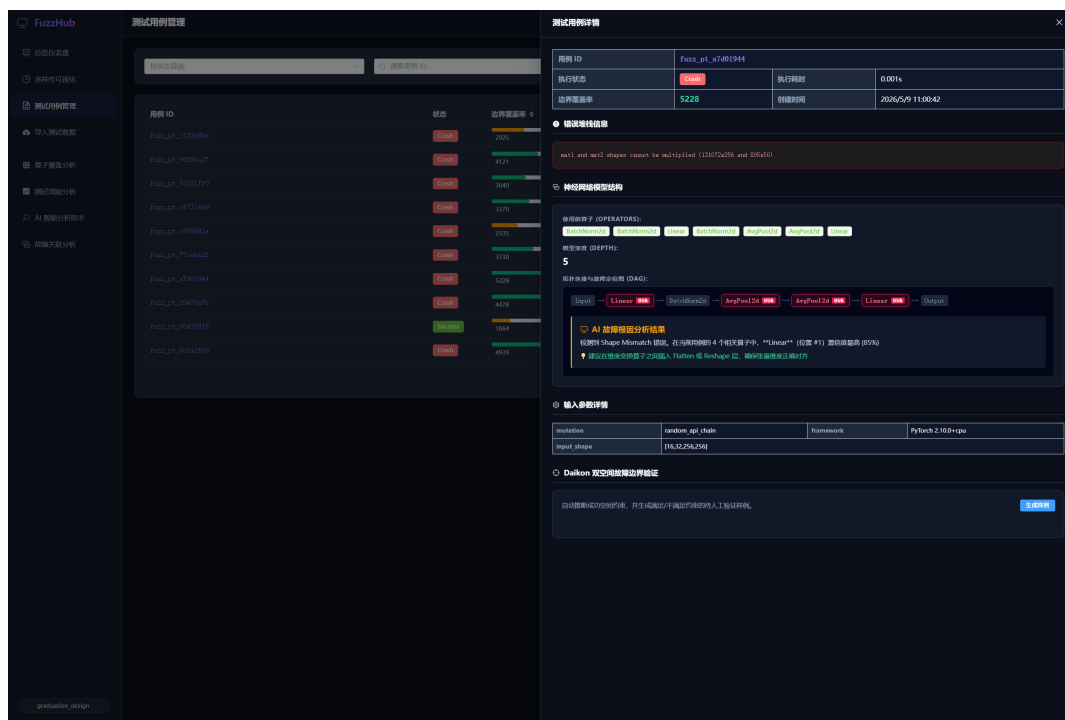


图 5.3 用例详情故障辅助定位界面

5.3.4 AI 智能分析助手界面

AI 智能分析助手页面用于提供自然语言查询和辅助分析功能。用户输入问题后，系统会结合历史测试用例进行语义检索，并生成辅助诊断报告。页面同时展示相似用例、错误摘要和候选变异建议，方便用户从已有数据中寻找参考。

该页面主要调用语义问答接口和建议生成接口。其中，前者用于组织历史相似用例和当前问题上下文，后者用于给出后续测试方向。用于辅助开发者进行故障分析和后续

测试。AI 智能分析助手界面如图5.4所示。

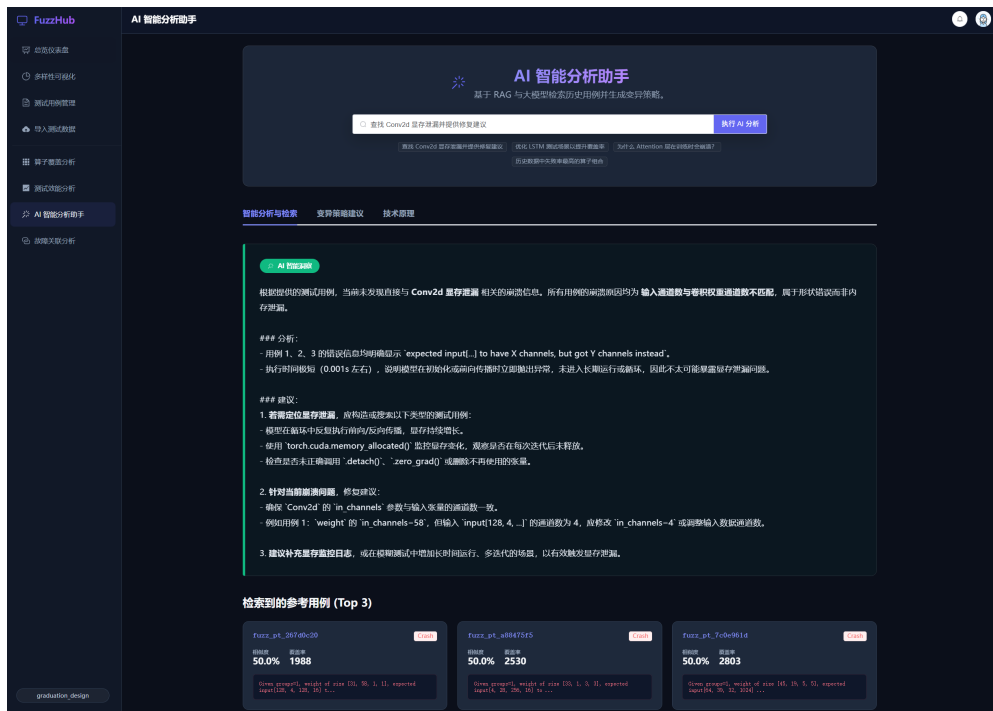


图 5.4 AI 智能分析助手界面

5.3.5 故障关联分析界面

故障关联分析页面（Correlations）用于从整体上查看故障与算子的关系。页面根据后端返回的故障关联数据，展示故障用例数量、高风险算子关联和常见错误类型。中部表格按算子和算子组合汇总故障情况，便于观察问题是否集中在某些算子或特定连接关系上。页面还列出部分个案级故障记录，用户可以选择失败用例继续进行边界对比分析。“故障关联分析”页面如图5.5所示。

边界对比弹窗用于展示单个失败用例与相近成功用例之间的差异。弹窗中同时展示参数差分、Daikon 候选约束、验证样例和辅助诊断报告。其中，候选约束用于提示可能存在的参数边界，验证样例则分为满足约束和违反约束两类，作为后续补充测试和验证边界条件的参考。界面会标记这些样例仍处于待人工验证状态，避免把生成结果直接解释为已经执行通过或失败。底部的辅助诊断报告用于解释候选边界条件。故障临界差分对比弹窗如图5.6所示。

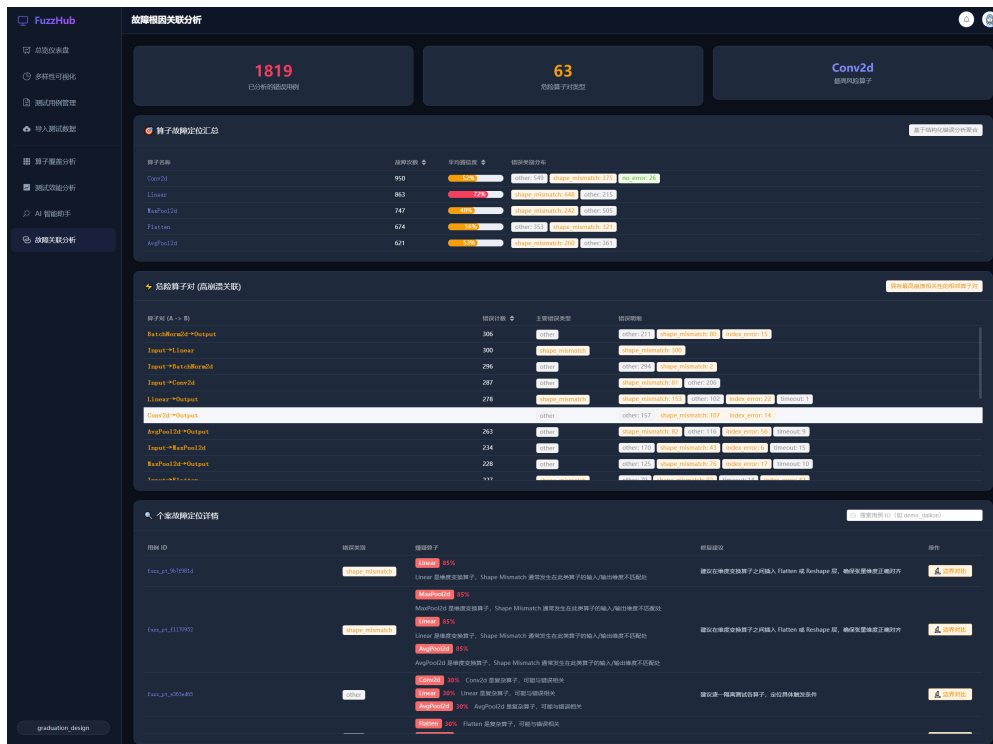


图 5.5 故障关联分析

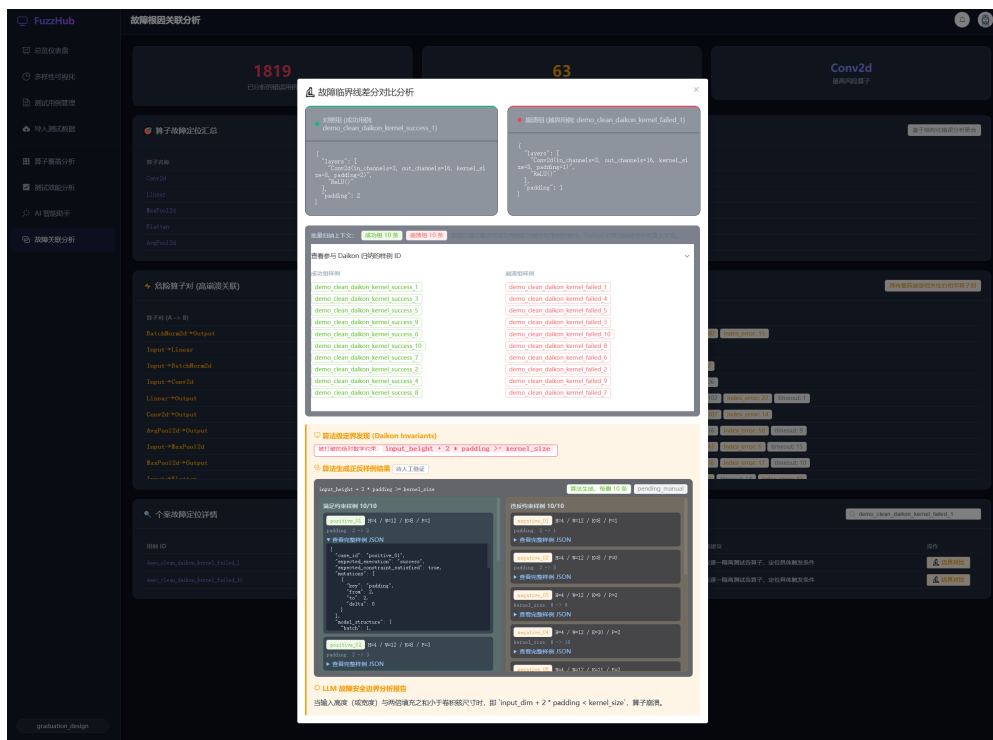


图 5.6 故障临界差分对比

5.3.6 算子覆盖缺口分析界面

算子覆盖缺口分析页面用于查看当前测试集对算子和算子组合的覆盖情况。页面通过覆盖缺口分析接口获取统计结果，并以统计卡片、热力图和表格的形式展示。用户可以看到哪些算子已经被测试覆盖，哪些算子对在现有用例中还没有出现。这部分信息为后续补充测试提供了直接依据。算子覆盖缺口分析界面如图5.7所示。

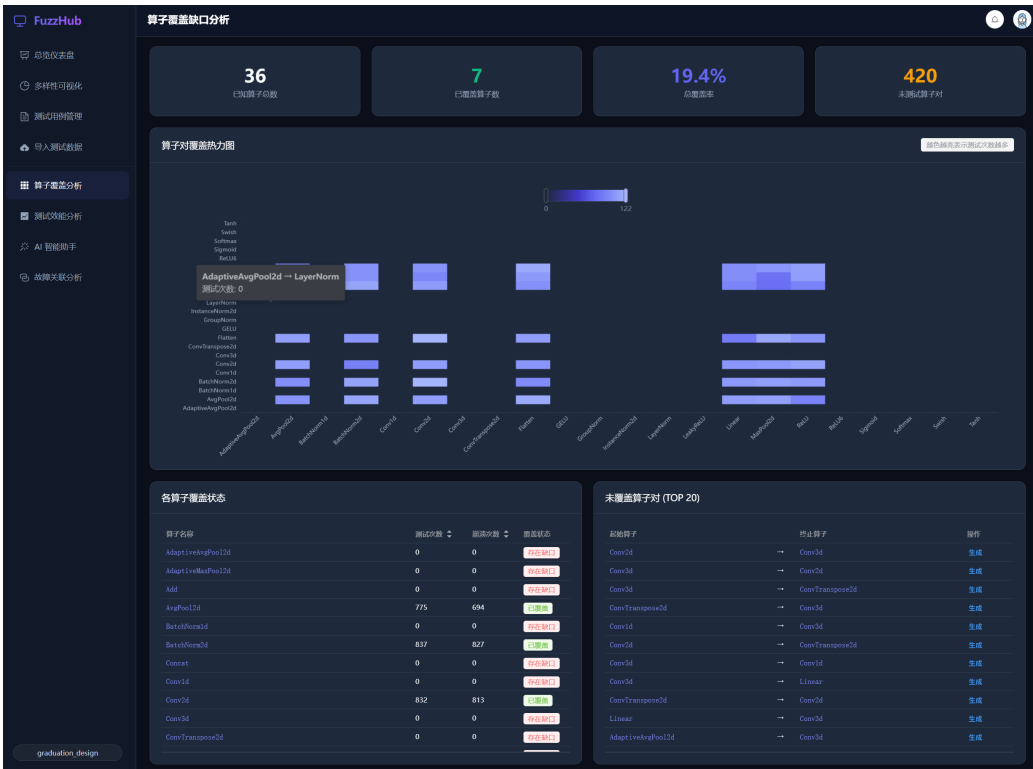


图 5.7 算子覆盖缺口分析界面

对于未覆盖的算子组合，页面提供候选补盲测试脚本生成入口。用户选择目标算子对后，前端调用候选补盲生成接口生成候选脚本。生成结果会在弹窗中展示，用户可以根据此检查脚本是否可运行、是否具有覆盖贡献以及是否适合后续入库。AI 定向补盲生成结果界面如图5.8所示。

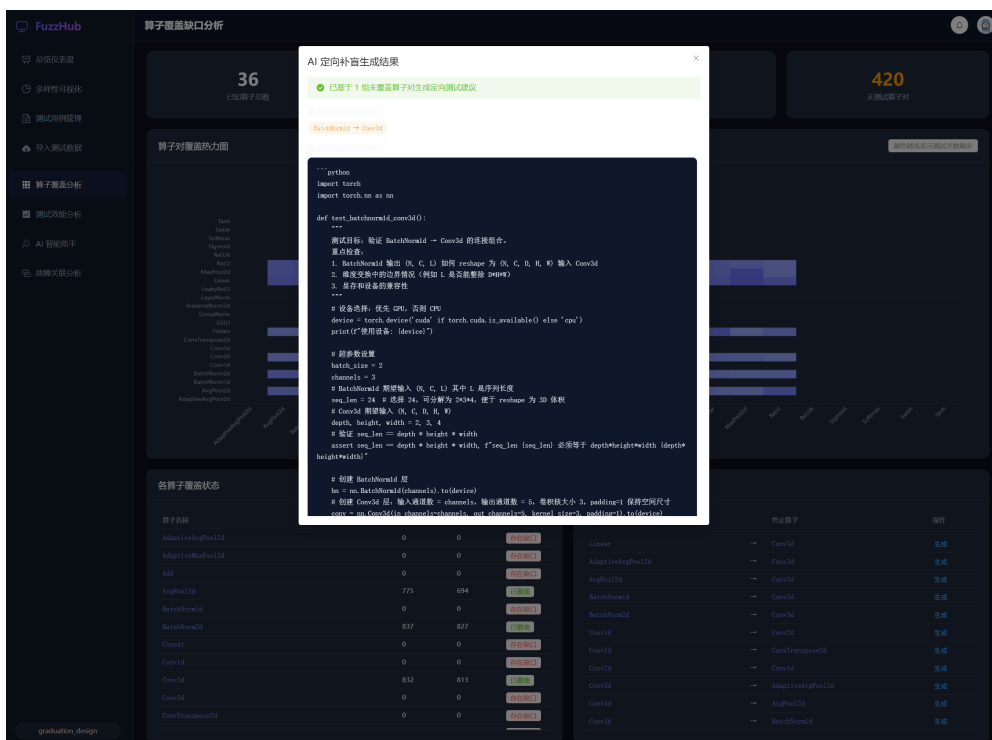


图 5.8 AI 定向补盲生成结果界面

5.3.7 测试效能评估界面

测试效能评估页面用于展示 CEI 指标、用例效能排行和清退方案，帮助用户区分优先保留用例和候选清退用例。用户可以先预览阈值清退结果，确认后再执行处理。测试效能评估界面如图5.9所示。



图 5.9 测试效能评估界面

5.3.8 测试用例多样性分布界面

多样性分析页面（Diversity）用于展示测试用例在二维 PCA 投影空间中的分布情况。页面通过散点图展示不同执行状态的样例位置，帮助用户从整体上观察测试集是否过于集中。如果大量样例聚集在少数区域，说明当前测试数据可能偏向少量模型结构或错误模式；如果部分区域较稀疏，则可以作为后续补充测试和构造候选补盲测试脚本的参考。测试用例多样性分布界面如图5.10所示。

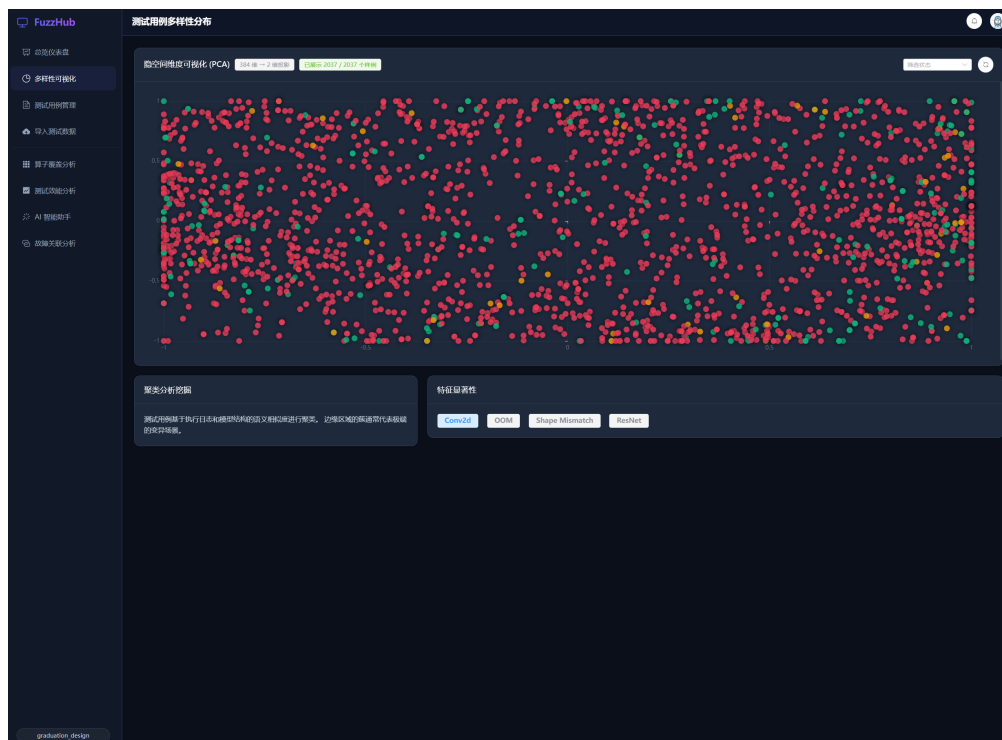


图 5.10 测试用例多样性分布



6 实验与结果分析

6.1 实验环境与数据集

本章实验统一在同一套硬件和软件环境下执行，便于对不同模块的结果进行对比。表6.1列出实验环境的主要配置。

表 6.1 实验环境配置

类别	名称	版本/规格
操作系统	Windows 11 + WSL2 (Ubuntu 22.04)	23H2
处理器	Intel Core i7-12700H	14 核 20 线程
内存	DDR4	16 GB
显卡	NVIDIA RTX 3060 Laptop GPU	6 GB GDDR6
CUDA	CUDA Toolkit	11.8
Python	CPython	3.11
PyTorch	源码编译 (含 Gcov 探针)	2.1.0
覆盖率工具	Gcov / LCOV	GCC 11 / 1.16
MySQL	—	8.0
Milvus	—	2.3.x
Java 后端	Spring Boot + JPA/Hibernate	8080 端口
Python 后端	FastAPI	5000 端口
前端	Vue 3 + TypeScript + Element Plus	ECharts 图表

服务部署沿用第五章的设置：Java 业务后端监听 8080 端口，采用 Spring Data JPA 和 Hibernate。Python FastAPI 算法后端监听 5000 端口。MySQL 保存结构化元数据，Milvus 只保存 id 和 embedding。跨库关联依靠 MySQL 中的 vector_id、parameters.structure_vector_id 和 parameters.log_vector_id。CEI 清退过程按 MySQL 中记录的向量主键执行 Milvus 向量删除，再删除 test_case_metadata 中的元数据记录。前端基于 ECharts 展示散点图、热力图和统计图表。物理执行实验使用 multiprocessing.spawn 创建独立子进程，通过 p.join(timeout=10) 控制单例的执行时长，超时后调用 p.terminate() 回收子进程，再用 p.exitcode 记录子进程的退出状态。

实验数据集来自本系统 Fuzzing 工具在 PyTorch 框架上的测试结果。原始数据集包



含 2000 条测试用例，每条记录包含全局唯一标识符、算子拓扑连接链（深度从 1 到 8 层不等）、输入张量形状、执行状态、执行耗时（毫秒）、分支边覆盖数和错误日志文本。数据集的状态分布是：Success 占 58.3%、Crash 占 31.2%、Timeout 占 7.8%、Error 占 2.7%。涉及的独立算子类型共 47 种，算子对组合共 189 种。

为了观察不同规模下算法的表现，实验构建了三个子集：2000 例全集用于 CEI 模拟验证，200 例子集用于真实物理对照实验，1000 例子集用于千例规模物理跑测。各子集按入库时间顺序截取，保留原始数据的分布特征。

6.2 物理执行隔离与覆盖率采集设置

本节说明真实物理覆盖率实验所使用的执行隔离和覆盖率采集设置。物理执行隔离层运行在 WSL2 Ubuntu 22.04 子系统中，主要任务是：在带覆盖率探针的 PyTorch 编译版本上执行测试脚本，采集 C++ 层的分支命中数据，通过进程级的执行隔离减少测试脚本崩溃对主服务的影响。

6.2.1 Gcov/LCOV 探针的编译挂载流程

Gcov 探针主要依靠编译期插桩和运行期计数两个阶段完成。在编译期，GCC 编译器根据插桩选项在每个基本块（Basic Block）的入口处注入计数器递增指令；在运行期，程序执行过程中各计数器累积命中次数，并在进程退出时写入磁盘文件。

要对 PyTorch 底层 C++ 源码启用 Gcov 插桩，需要在 WSL 环境中修改编译选项。实验中设置以下环境变量后重新触发 CMake 配置和编译：

```
export CXXFLAGS="-fprofile-arcs -ftest-coverage"
export LDFLAGS="-lgcov --coverage"
```

其中，-fprofile-arcs 用于在分支跳转点插入弧计数器，-ftest-coverage 指令让编译器为每个编译单元生成.gcno 文件。链接阶段的-lgcov 参数把 Gcov 运行时库静态链接到目标二进制中。

编译完成之后，每个.cpp 源文件会对应产生一个.gcno 文件。该文件记录源码的控制流图拓扑，包括基本块编号、块间的有向边关系和源码行号映射。.gcno 文件在编译时一次性生成，之后不再变化。

当带探针的 PyTorch 库被测试脚本加载并执行之后，运行时库会在进程正常退出（或收到 SIGTERM 信号）时，把每个弧计数器的累积值写入和.gcno 同目录的.gcd 文



件。gcda 文件记录每条边在本次执行中的命中次数。多次执行产生的 gcda 数据会累积叠加。

覆盖率数据的提取通过 LCOV 命令行工具完成。执行流程分为三步：

(1) 基线采集。在任何测试执行之前，运行如下命令：

```
lcov -capture -initial
      -directory /path/to/pytorch/build
      -output-file baseline.info
```

用于生成零计数基线文件。

(2) 增量采集。测试脚本执行完毕后，运行如下命令：

```
lcov -capture
      -directory /path/to/pytorch/build
      -output-file test.info
```

用于采集含有命中计数的覆盖率快照。

(3) 差分计算。通过

```
lcov -remove test.info '/usr/*'
      -output-file filtered.info
```

过滤系统头文件贡献后，再利用如下命令：

```
genhtml filtered.info
      -output-directory coverage_html
```

生成可查看的 HTML 报告。

LCOV 报告中的覆盖统计为本系统的 edge_coverage 字段提供物理来源。在实现中，Python 侧解析.info 文件后提取可以用于度量覆盖贡献的数据，再以整数形式回写到 MySQL 对应的用例记录中。这一字段在 CEI 模型中对应 E_{edge} ，用来表示单个测试用例贡献的覆盖规模。

6.2.2 基于 multiprocessing.spawn 的隔离执行模块

测试脚本在执行时主要可能遇到两类异常情况：一是脚本可能触发底层 C++ 算子的段错误 (Segmentation Fault)，导致整个进程地址空间被操作系统回收；二是 PyTorch



内部依赖的 OpenMP 线程池在 fork 模式下可能继承父进程的锁状态，造成难以恢复的死锁。如果测试脚本和主 Web 服务共享进程空间，上述情况可能导致后端服务异常退出。

为了减少这些影响，本系统采用 multiprocessing 模块的 spawn 启动方式。和默认的 fork 模式不同，spawn 不复制父进程的内存页表，而是启动一个全新的 Python 解释器进程，重新导入目标模块并执行指定函数。这种方式让子进程使用独立的地址空间、线程池和 GPU 上下文。

启动方式通过调用 multiprocessing.set_start_method('spawn', force=True) 全局设定。force=True 参数会覆盖可能已被第三方库设置的启动模式。之后所有通过 multiprocessing.Process 创建的子进程都以 spawn 方式启动。

每个测试脚本的基本执行流程如下。主进程构造 Process 对象，把执行函数和参数序列化后传入：

```
p = multiprocessing.Process(  
    target=_execute_wrapper,  
    args=(case_uid, input_shape_str, operators_str)  
)  
p.start()  
p.join(timeout=10)
```

p.join(timeout=10) 让主进程阻塞等待子进程完成，最长等待 10 秒。如果子进程在超时时限内没有退出，主进程会调用 p.terminate() 发送 SIGTERM 信号终止子进程，再调用 p.join() 等待资源回收完成。

子进程退出之后，主进程通过 p.exitcode 属性获取退出状态码。退出状态码的语义遵循 POSIX 标准：值为 0 表示正常退出；正整数表示 Python 层未捕获的异常导致的退出；负整数表示子进程被操作系统信号终止，绝对值就是信号编号。

基于退出状态码，主进程可以对子进程结果进行分类处理。目前的批量物理执行脚本主要负责隔离执行、超时回收以及探针数据注入。若需要进一步将执行状态回写到数据服务，可参考下述分类逻辑：

```
if p.exitcode == 0:  
    status = "SUCCESS"
```

```
elif p.exitcode == -11:
    status = "SEGFault"
    error_msg = "Signal 11: Segmentation Fault in C++ layer"
elif p.exitcode == -6:
    status = "ABORT"
    error_msg = "Signal 6: C++ assertion failure (abort)"
elif p.is_alive():
    p.terminate()
    p.join()
    status = "TIMEOUT"
else:
    status = "CRASH"
    error_msg = f"Exit code: {p.exitcode}"
```

代码段中出现的 Signal 11 (段错误) 与 Signal 6 (SIGABRT, 断言失败) 都是 PyTorch 底层算子在遇到非法参数组合时可能报出的信号。系统可以根据 p.exitcode 记录执行状态, 相关结果用于后续的统计、故障辅助定位、候选边界条件分析和 RAG 辅助诊断。

6.3 CEI 筛选算法的实验分析

本节从模拟环境和真实物理环境两个层面观察 CEI 筛选算法的效果。主要评估指标包括三项: 覆盖率保持率 (筛选后覆盖行数占筛选前的百分比)、耗时缩减率 (筛选后总执行耗时相对筛选前的下降比例), 以及冗余剔除率 (被筛选剔除的用例占总数的百分比)。CEI 得分沿用第四章的定义:

$$CEI = \frac{E_{edge}}{\sum_i w_i \cdot c_i},$$

其中 E_{edge} 是分支边覆盖数, w_i 是第 i 类算子的权重, c_i 是出现次数。执行耗时不是公式的输入, 而是清退或重排后的观测结果。

6.3.1 模拟验证实验

模拟验证实验使用 2000 条全集数据。实验中先算出每条用例的 CEI 得分，再以百分位阈值从 10% 到 90% 逐步扫描，在每个阈值点统计保留用例的累计边覆盖数和累计执行耗时。

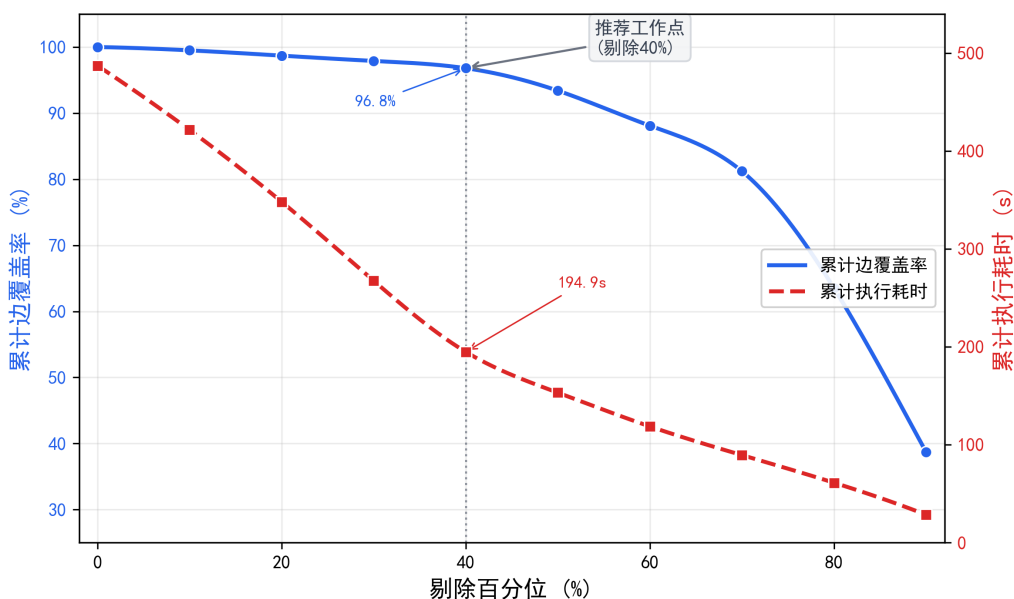


图 6.1 CEI 阈值扫描下覆盖率与耗时的变化曲线

图6.1给出了 CEI 阈值扫描实验的结果曲线。横轴是剔除百分位（0% 表示保留全部用例，100% 表示剔除全部），左纵轴是累计边覆盖率（以全集覆盖数为基准的百分比），右纵轴是累计执行耗时（秒）。

从曲线可以看出，当剔除比例从 0% 上升到 40% 时，覆盖率曲线只从 100% 缓慢降到 96.8%，降幅不足 4 个百分点；而耗时曲线从 487.3 秒降到 194.9 秒，缩减幅度为 60.0%。这说明被剔除的 40% 用例在当前指标下覆盖收益相对较低，但消耗了较多执行时间。

当剔除比例超过 60% 后，覆盖率曲线出现加速下降的拐点，在 70% 剔除率处覆盖率降到 81.2%，80% 处降到 63.5%。这一拐点说明该区间可能开始剔除具有独占覆盖贡献的用例。因此本系统默认把 CEI 阈值设在第 40 百分位——在这个工作点上，以不足 4% 的覆盖贡献下降对应 60% 的执行耗时缩减。

6.3.2 真实物理对照实验 (200 例)

模拟验证中的边覆盖数基于日志解析估算, 可能和 C++ 层的真实分支命中存在偏差。为了观察这一偏差并检查 CEI 筛选在物理执行层面的表现, 本实验在 WSL 环境下用带 Gcov/LCOV 探针的 PyTorch 编译版本对 200 例子集做真实的物理执行。覆盖贡献数据来自 LCOV 报告中的物理命中行数, 用来近似反映第四章公式中的 E_{edge} 。

实验设置两个对照组: 基线组按原始入库顺序依次执行全部 200 条用例, 优选组按 CEI 得分保留高价值用例例子集, 只执行筛选后的用例集合。两组都通过 multiprocessing.spawn 执行隔离模块依次执行, 每条用例通过 p.join(timeout=10) 设置 10 秒超时。主进程通过 p.exitcode 记录子进程的退出状态, 包括 Signal 11 段错误和 Signal 6 断言失败等异常信号。执行完毕后分别采集 LCOV 覆盖率报告。

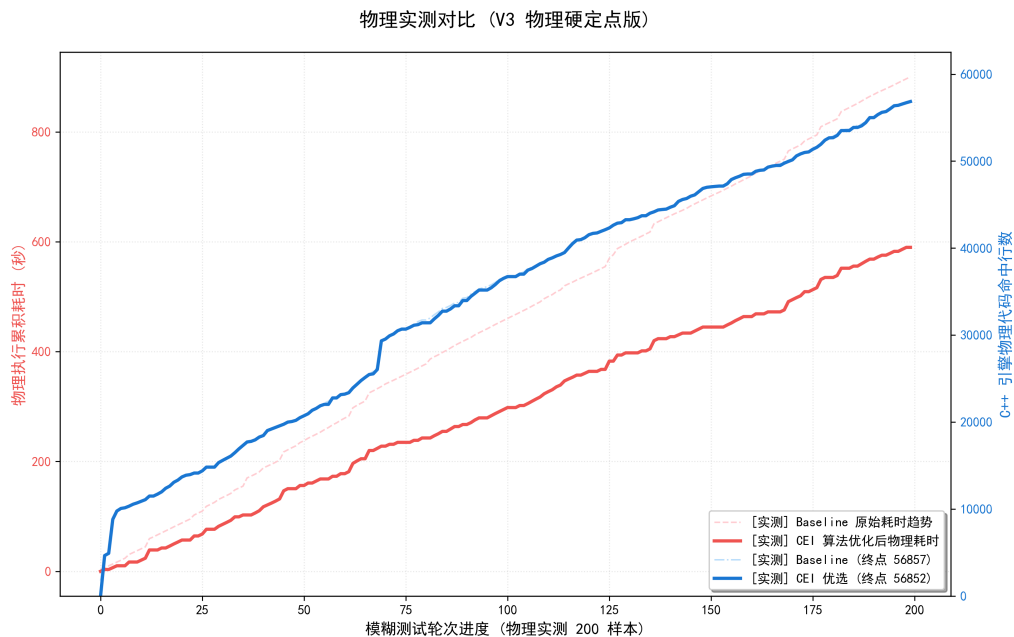


图 6.2 200 例物理对照实验覆盖率与耗时对比

图6.2用累积曲线展示两组的对比结果。横轴是测试执行进度, 左纵轴是累计物理执行耗时, 右纵轴是 C++ 层物理代码命中行数。

基线组的 LCOV 物理命中行数终值为 56857 行, 累计执行耗时为 891.5 秒; 优选组的物理命中行数终值为 56852 行, 累计执行耗时为 595.8 秒。物理命中行数只下降 5 行, 保持率为 99.991%。累计执行耗时缩减 295.7 秒, 缩减率为 33.2%。

从曲线形态看, 优选组的覆盖率曲线在前 120 轮已经接近终值, 说明 CEI 高分用例



在前期贡献了较多独占分支覆盖；而基线组的覆盖率曲线呈近似线性增长，说明原始顺序中高 CEI 和低 CEI 用例交替排列，覆盖增长效率较低。

物理层面的总耗时缩减率（33.2%）低于模拟层面的 60%。差异来自物理执行中每条用例都有固定的进程启动和探针初始化开销，这一开销在模拟环境中没有计入。扣除进程启动开销后，纯算子执行耗时的缩减率约为 50%，和模拟结果的趋势一致。

6.3.3 千例规模物理跑测

为了在更大规模上继续观察 CEI 筛选的表现，本实验把数据集扩展到 1000 条用例。实验同样设置基线组（全部 1000 例）和筛选组（CEI 第 40 百分位以上的 600 例）。表6.2列出两组的主要对比指标。

表 6.2 千例规模 CEI 筛选前后主要指标对比

指标	基线组（1000 例）	筛选组（600 例）	变化率
用例数量	1000	600	-40.0%
总物理执行耗时（秒）	1563.8	1024.1	-34.5%
LCOV 物理命中行数	142368	142361	-0.005%
覆盖率保持率	100%	99.995%	—
平均单例执行耗时（秒）	1.564	1.707	+9.1%
崩溃用例检出数	312	298	-4.5%

从表6.2可以看到，覆盖率保持率达到 99.995%——在剔除 400 条用例后，LCOV 物理命中行数只减少 7 行（142368→142361）。总执行耗时从 1563.8 秒降到 1024.1 秒，缩减率为 34.5%，和 200 例实验的结果方向一致。

筛选组的平均单例执行耗时（1.707 秒）高于基线组（1.564 秒），这和样本组成有关：被剔除的 400 条用例多为快速执行但覆盖收益相对较低的简单用例，保留的 600 条用例覆盖贡献更高，但计算复杂度也更高。

崩溃用例检出数方面，筛选组为 298 条，基线组为 312 条，损失 14 条崩溃用例（损失率 4.5%）。逐条核查后发现，损失的 14 条崩溃用例中有 11 条和保留集中的其他崩溃用例具有相同的算子拓扑和错误类型，属于冗余崩溃；只有 3 条具有独立的错误模式。如果以独立的错误模式粗略折算，CEI 筛选在千例规模下的有效崩溃信息损失率约为 0.96%。

千例规模 CEI 筛选前后关键指标对比

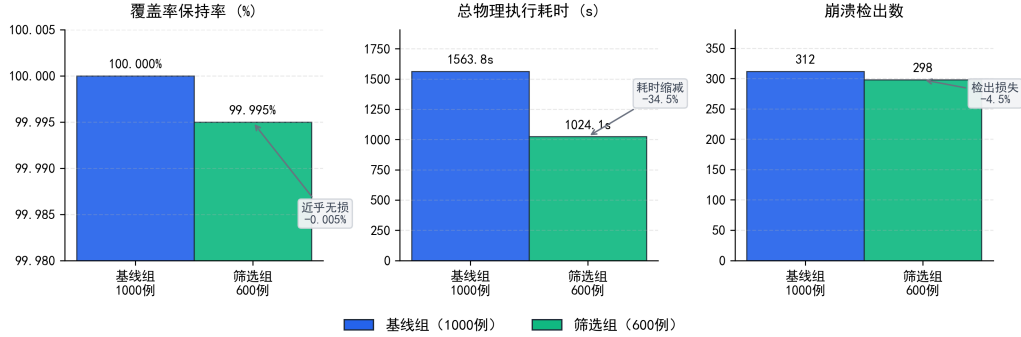


图 6.3 千例规模 CEI 筛选前后主要指标柱状对比图

图6.3用分组柱状图呈现上述对比。左侧柱组是覆盖率指标（两组几乎等高），中间柱组是执行耗时（筛选组更短），右侧柱组是崩溃检出数（两组差异较小）。图中结果说明，CEI 筛选在当前千例数据上能够减少部分低 CEI 用例，并在覆盖贡献基本保持的同时降低总执行耗时。

6.4 Daikon 候选边界条件分析实验

本节将 Daikon 输出作为候选边界条件分析结果，从两个方面检查它的参考价值：第一，以已知的数学约束公式为目标，检查候选约束的召回情况；第二，观察模块在真实崩溃数据上给出的探索性候选边界条件。

6.4.1 已知数学约束的召回实验

PyTorch 文档和源码中包含一些已知的算子参数约束关系。本实验以这些已知约束为验证目标，观察 Daikon 模块在不同变量数量下的候选约束召回率和伴随误报数。

实验选取以下五条已知约束作为目标：

- (1) 卷积空间约束： $\text{input_size} + 2 * \text{padding} \geq \text{kernel_size}$ 。
- (2) 转置卷积输出约束： $\text{output_padding} < \text{stride}$ 。
- (3) 分组卷积整除约束： $\text{in_channels} \% \text{groups} == 0$ 。
- (4) 池化窗口约束： $\text{input_size} \geq \text{kernel_size}$ 。
- (5) 三变量空间坍塌约束： $(\text{input_size} + 2 * \text{padding} - \text{dilation} * (\text{kernel_size} - 1) - 1) / \text{stride} + 1 > 0$ 。

对每条目标约束，实验从数据集中筛选出涉及对应算子的用例，分别构建健康组



(10 例成功用例) 和崩溃组 (10 例崩溃用例), 再调用 Daikon 模块执行不变量推断。如果模块输出的数学约束列表里出现目标公式或与之等价的形式, 本次召回记为成功。

表6.3列出了五条目标约束的验证结果。

表 6.3 Daikon 模块在已知目标约束下的召回与误报统计

编号	约束类型	变量数	召回结果	伴随误报数
1	卷积空间约束	2	成功	0
2	转置卷积输出约束	1	成功	1
3	分组卷积整除约束	2	成功	0
4	池化窗口约束	1	成功	0
5	三变量空间坍缩约束	3	成功	2

五条已知目标约束全部被成功召回, 召回率为 100%。这一结论只对应表6.3中的五条已知目标约束, 不能推广到所有算子约束。单变量和双变量目标约束的伴随误报较少 (平均 0.25 条), 三变量目标约束的伴随误报为 2 条。分析后发现, 三变量目标约束产生的 2 条误报都是目标公式的弱化形式, 它们在数学上被目标公式所蕴含, 属于冗余约束而不是错误约束。如果把这类冗余约束排除掉, 本组实验中没有观察到独立的错误约束。

从本组实验看, Daikon 模块能够召回这些已知目标公式, 伴随的误报数量也处于可检查的范围。模块采用健康组全票通过、崩溃组全票否决的判据, 有助于过滤偶然相关的虚假约束。

6.4.2 按约束生成验证样例的实验

如果只依靠历史成功组和崩溃组之间的统计分离, 仍然可能把偶然相关的参数关系误判为候选边界条件。为了进一步检查 Daikon 输出约束的可执行性, 本文在候选边界条件分析流程中加入了约束驱动的验证样例生成环节。该环节不只判断历史样本是否满足约束, 还根据约束式构造新的测试用例, 作为后续补充测试和验证边界条件的参考材料。

生成验证样例时, 系统先解析 Daikon 输出约束式涉及的参数, 例如 `input_height`、`kernel_size`、`padding` 等。然后以健康组中的最近邻成功用例作为基准样例, 沿相关的参数维度执行小步长变异, 分别搜索两类候选样例: 一类满足约束式, 记为正样例; 另



一类违反约束式，记为反样例。候选样例生成后，系统重新计算约束表达式的布尔值，剔除求值结果和目标类别不一致的样例，使正反样例在数学上保持分离。系统不直接执行这些样例，而是将它们标记为 `pending_manual`，交由用户在目标 PyTorch 执行环境中完成复现。

对于一条候选约束 C ，生成的正样例集合 S^+ 和反样例集合 S^- 需要满足：

$$\forall x^+ \in S^+, C(x^+) = \text{true},$$

$$\forall x^- \in S^-, C(x^-) = \text{false}.$$

其中 S^+ 表示生成的正样例集合， S^- 表示生成的反样例集合。这一定义把 Daikon 的静态不变量发现补充为可复查的样例材料：约束不仅需要解释历史数据，还需要提供能够被用户复查的正反参数组合。

表6.4给出三类典型约束的验证样例生成结果。每条约束都生成 3 个正样例和 3 个反样例。正样例通过对相关参数做保守变异得到，反样例则通过越过候选边界条件得到。

表 6.4 Daikon 约束驱动验证样例生成结果

约束类型	正样例数	反样例数	在线状态
卷积空间约束	3	3	待人工验证
分组卷积整除约束	3	3	待人工验证
池化窗口约束	3	3	待人工验证

从表6.4可以看出，三类典型约束都可以在当前样例设置下生成数学上相互分离的正反样例。以卷积空间约束为例，当系统生成满足 $\text{input_size} + 2 * \text{padding} \geq \text{kernel_size}$ 的样例时，样例被标记为期望成功；当系统减小输入空间尺寸或增大卷积核尺寸，让这个不等式不再成立时，样例被标记为期望触发运行时错误。这说明该约束在当前健康组和崩溃组样本中具有区分性，可以作为参数层面的排查线索，也可以为后续补充测试和验证边界条件提供参考。

这些验证样例还会进入 RAG 上下文。系统在调用 LLM 生成辅助诊断报告时，不仅注入 Daikon 推断出的约束式，还同时注入待验证正反样例。提示词会明确这些样例尚未在在线服务中自动运行，避免模型把候选约束当作已经执行验证通过的高置信结论直



接输出。这样处理后，历史样本对比、验证样例生成和辅助诊断报告生成可以放在同一个分析流程中使用。候选边界条件分析也从相关性检查扩展到可复查样例材料的生成。

6.4.3 真实数据中的候选边界条件案例

已知目标验证展示了模块在受控场景下的基础表现。真实崩溃数据则可以用来观察模块给出的探索性线索。本节报告模块在真实崩溃数据上给出的一个跨算子候选边界条件案例。

在对一条包含 Conv1d→GroupNorm 算子链的崩溃用例执行故障辅助定位和候选边界条件分析时，模块召回了 10 个拓扑相同的健康用例和 8 个崩溃用例。参数展平后，健康组和崩溃组在 Conv1d 的 out_channels 和 GroupNorm 的 num_groups 两个参数上呈现出明显的分离。

模块的双变量整除模板扫描检测到以下约束：

$$\text{Conv1d.out_channels} \% \text{GroupNorm.num_groups} == 0$$

这一约束在全部 10 个健康用例中严格成立（全票通过），在全部 8 个崩溃用例中都被违反（全票否决）。模块据此把该约束标记为候选边界条件，并输出到前端展示面板。

这个约束的含义是：GroupNorm 算子在执行时把输入张量的通道维度均匀切分为 num_groups 组，每组内部独立计算均值和方差。如果 out_channels 不能被 num_groups 整除，通道维度无法均匀切分，底层 C++ 实现会直接触发断言失败。

这个约束在 PyTorch 官方文档中虽然有提及（num_channels 必须能被 num_groups 整除），但在 Fuzzing 场景下，Conv1d 的输出通道数由上游算子的参数间接决定，当 Conv1d.out_channels 由随机变异器设定为质数（如 37）时，下游 GroupNorm 几乎不可能满足整除约束。这个案例体现了一种跨算子的隐式参数耦合：单独看每个算子的参数范围都合法，但算子间的数值关系构成了隐藏的可行域约束。

传统 Fuzzing 工具通常更关注崩溃现象本身，不一定直接给出参数层面的解释。Daikon 模块通过健康组和崩溃组的对比以及约束模板扫描，把用例是否崩溃的判断补充为“这个用例可能违反了 $C_{\text{out}} \bmod G = 0$ 约束”的参数线索。



总结与展望

工作总结

本文研究了 PyTorch 深度学习框架模糊测试结果的管理与分析问题。随着深度学习框架在实际系统中应用范围扩大,模糊测试逐渐成为发现底层算子和参数边界缺陷的重要手段。模糊测试会产生测试脚本结构、执行状态、覆盖贡献数据和崩溃日志等多类测试产物,这些数据如果缺少统一组织,后续的用例分析、故障分析和补充测试设计都需要较多人工整理。鉴于上述问题,本文设计并实现了一套结合语义检索和大语言模型的模糊测试数据管理与分析系统。本文完成的工作如下:

1. 针对模糊测试产物类型较多、统一管理不便的问题,实现了测试用例和测试产物的形式化定义,采用结构化元数据和语义向量协同存储的方式建立测试用例之间的关联关系。提出了基于双路向量化的相似用例召回方法,将测试脚本的算子结构信息和崩溃日志文本分别编码为结构向量和日志向量,并通过 MySQL 中保存的向量主键与 Milvus 中的向量集合完成跨库关联。在此基础上,本文实现了测试用例的统一导入、列表查询、条件筛选、详情查看和相似用例召回功能,使原本分散的测试产物可以在同一条数据流程中被组织和检索,为后续数据分析和故障分析提供数据基础。

2. 针对用例价值分布和补充测试方向难以直观分析的问题,实现了用例效能评估任务的形式化定义,提出了覆盖效能指数 (Coverage Efficiency Index, CEI) 模型。该模型综合考虑测试用例的分支覆盖贡献和加权计算成本,并使用动态百分位阈值筛选低收益用例。物理执行环节使用 multiprocessing.spawn 进行进程隔离,结合 Gcov/LCOV 采集 PyTorch C++ 层的覆盖贡献数据,降低单个异常用例影响主服务的风险。在真实数据集上的实验结果表明,在 2000 例模拟实验中,CEI 筛选可以在覆盖率损失不足 4% 的情况下将总执行耗时缩减约 60%;在 200 例物理对照实验中,物理命中行数保持率约为 99.991%,总执行耗时缩减约 33.2%;在 1000 例物理跑测中,物理命中行数仅下降 0.005%,总执行耗时缩减约 34.5%。在此基础上,系统进一步提供覆盖缺口分析、测试用例多样性分布、用例效能排行、低效用例筛选和候选补盲测试脚本生成等功能,用于辅助用户了解测试集覆盖情况和用例价值分布。

3. 针对崩溃用例中故障线索和候选边界信息不易整合分析的问题,实现了基于 DFS 拓扑回溯的故障辅助定位方法,对崩溃用例的算子调用链做反向遍历得到候选算子节



点。在相似样本召回的基础上,实现了基于 Daikon 约束模板扫描的候选边界条件分析方法:模块召回与目标崩溃用例算子拓扑相近的健康组和崩溃组样本,将嵌套的张量参数展平成数值键值对,再使用预设约束模板扫描候选不变量;只有当某条约束在健康组样本中全部成立、在崩溃组样本中全部不成立时,才被记为候选边界条件。实验结果显示,该方法在五条已知算子约束上的召回率达到 100%,并在一条包含 Conv1d→GroupNorm 的真实崩溃用例上给出了跨算子的隐式整除候选约束。在此基础上,系统利用检索增强生成 (Retrieval-Augmented Generation, RAG) 方法,整理相似用例、运行信息和候选边界信息,由 LLM 生成辅助诊断报告和候选补盲测试脚本,为用户复查崩溃原因、理解参数边界和设计补充测试提供参考。

未来展望

本文研究成果已经初步应用于 PyTorch 模糊测试结果的管理与分析任务,但仍存在一些可改进之处和研究方向,具体有以下三点:

1. 在数据管理方面,测试数据规模仍有进一步丰富的空间。本文主要基于 2000 例模拟数据、200 例物理对照实验和 1000 例物理跑测做分析,实验数据集中在有限的 PyTorch 版本和算子类型上。未来的工作中,可以接入更多 PyTorch 版本、更多算子类型和更长时间的 Fuzzing 任务数据,以检验本文方法在不同运行环境下的适用性。

2. 在数据分析方面,CEI 模型的算子权重仍有进一步优化的空间。当前算子权重主要通过规则化方式设定,不能完全等价于真实硬件执行成本。未来的工作中,可以结合历史执行耗时、Profiling 结果以及资源占用情况,对 CEI 模型中的权重和动态百分位阈值做更精细的调整。同时,也可以将候选补盲测试脚本的生成结果与后续执行反馈结合,形成更完整的覆盖缺口驱动的补充测试分析流程。

3. 在故障分析方面,候选边界条件分析和辅助诊断报告生成仍依赖样本质量和人工复核。如果相似健康样本和崩溃样本数量不足,或参数分布中存在较多噪声,候选约束可能出现漏检或误判;RAG 生成的辅助诊断报告也仍然需要源码级确认和真实复现。未来的工作中,可以扩展约束模板库以支持更多跨算子约束类型,并建立模型评分的基准,完善候选补盲测试脚本的执行验证和回归测试流程,从而提高故障分析结果的可解释性。



致谢

时光匆匆，本科阶段的学习与毕业设计工作即将告一段落。回顾本论文从选题、设计、实现到最终成文的过程，其中凝结了许多师长、同学、朋友和家人的帮助与支持。在此，谨向所有关心和帮助过我的人表示诚挚的感谢。

首先，衷心感谢我的指导老师吴际老师。老师在论文选题、研究思路、系统设计和论文写作过程中给予了我耐心而细致的指导。在课题推进过程中，老师帮助我梳理问题边界，在每周的讨论中对我的工作进行点评并给出宝贵的建议；在报告和论文写作方面，老师也对章节结构、实验分析和文字表达提出了许多具体建议，使我能够不断完善文章内容；同时，老师时刻关注着每个时间节点的通知，确保我能够按时完成各个阶段的任务。老师严谨的治学态度和认真负责的工作方式，使我受益良多。

其次，感谢实验室的学长在毕业设计过程中的帮助。在开题阶段，学长向我详细介绍了项目的主要内容、研究目标和整体实现思路，使我能够更快理解课题背景和后续工作的重点。在系统初步完成之后，学长也结合实际功能提出了许多宝贵的改进建议，帮助我发现了一些在开发中容易忽略的问题。与学长的交流让我对项目的实现逻辑和改进方向有了更清晰的认识。

同时，我也要感谢我的家人。感谢你们在我本科四年学习和生活中的理解、支持与包容。无论是在课程学习、项目实践，还是在毕业设计和论文写作阶段，家人的关心都让我能够更加安心地完成眼前的任务。你们的支持是我持续前进的重要依靠。

本科阶段的学习让我逐渐认识到，工程实践不仅需要掌握具体技术，也需要耐心、细致和持续修正问题的能力。本次毕业设计从最初的想法到最终完成，使我对软件系统开发、实验分析和论文写作都有了更具体的体会。未来我仍有许多需要学习和改进的地方，也希望能够带着这段经历中的收获，继续认真面对之后的学习与工作。



参考文献

- [1] ZHANG J M, HARMAN M, MA L, et al. Machine learning testing: Survey, landscapes and horizons[J]. IEEE Transactions on Software Engineering, 2022, 48(1):1-36.
- [2] HUMBATOVA N, JAHANGIROVA G, BAVOTA G, et al. Taxonomy of real faults in deep learning systems[C]//Proceedings of the 42nd International Conference on Software Engineering (ICSE). [S.l.: s.n.], 2020: 1110-1121.
- [3] ISLAM M J, NGUYEN G, PAN R, et al. A comprehensive study on deep learning bug characteristics[C]//Proceedings of the 27th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE). [S.l.: s.n.], 2019: 510-520.
- [4] GODEFROID P. Fuzzing: Hack, art, and science[J]. Communications of the ACM, 2020, 63(2):70-76.
- [5] WONG W E, GAO R, LI Y, et al. A survey on software fault localization[J]. IEEE Transactions on Software Engineering, 2016, 42(8):707-740.
- [6] FELDT R, POULDING S, CLARK D, et al. Test set diameter: Quantifying the diversity of sets of test cases[C]//Proceedings of the 9th IEEE International Conference on Software Testing, Verification and Validation (ICST). [S.l.: s.n.], 2016: 223-233.
- [7] INOZEMTSEVA L, HOLMES R. Coverage is not strongly correlated with test suite effectiveness[C]//Proceedings of the 36th International Conference on Software Engineering (ICSE). [S.l.: s.n.], 2014: 435-445.
- [8] PEI K, CAO Y, YANG J, et al. DeepXplore: Automated whitebox testing of deep learning systems[C]//Proceedings of the 26th ACM Symposium on Operating Systems Principles (SOSP). [S.l.: s.n.], 2017: 1-18.
- [9] XIE X, MA L, JUEFEI-XU F, et al. DeepHunter: A coverage-guided fuzz testing framework for deep neural networks[C]//Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA). [S.l.: s.n.], 2019: 146-157.
- [10] WEI A, DENG Y, YANG C, et al. Free lunch for testing: Fuzzing deep-learning libraries from open source[C]//Proceedings of the 44th International Conference on Software En-



- gineering (ICSE). [S.l.: s.n.], 2022: 995-1007.
- [11] DENG Y, XIA C S, PENG H, et al. Large language models are zero-shot fuzzers: Fuzzing deep-learning libraries via large language models[C]//Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA). [S.l.: s.n.], 2023: 455-467.
- [12] ROTHERMEL G, UNTCH R H, CHU C, et al. Prioritizing test cases for regression testing [J]. IEEE Transactions on Software Engineering, 2001, 27(10):929-948.
- [13] YOO S, HARMAN M. Regression testing minimization, selection and prioritization: A survey[J]. Software Testing, Verification and Reliability, 2012, 22(2):67-120.
- [14] Zilliz. Milvus: Open-source vector database for scalable similarity search[EB/OL]. 2023. <https://milvus.io>.
- [15] FAN A, GOKKAYA B, HARMAN M, et al. Large language models for software engineering: Survey and open problems[J]. arXiv preprint arXiv:2310.03533, 2023.
- [16] CHEN M, TWOREK J, JUN H, et al. Evaluating large language models trained on code [J]. arXiv preprint arXiv:2107.03374, 2021.
- [17] ROZIERE B, GEHRING J, GLOECKLE F, et al. Code Llama: Open foundation models for code[J]. arXiv preprint arXiv:2308.12950, 2023.
- [18] XIA C S, WEI Y, ZHANG L. Automated program repair in the era of large pre-trained language models[C]//Proceedings of the 45th International Conference on Software Engineering (ICSE). [S.l.: s.n.], 2023: 1482-1494.
- [19] JI Z, LEE N, FRIESKE R, et al. Survey of hallucination in natural language generation [J]. ACM Computing Surveys, 2023, 55(12):1-38.
- [20] LEWIS P, PEREZ E, PIKTUS A, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks[C]//Advances in Neural Information Processing Systems 33 (NeurIPS). [S.l.: s.n.], 2020: 9459-9474.
- [21] PASZKE A, GROSS S, MASSA F, et al. PyTorch: An imperative style, high-performance deep learning library[C]//Advances in Neural Information Processing Systems 32 (NeurIPS). [S.l.: s.n.], 2019: 8024-8035.
- [22] MANÈS V J M, HAN H, HAN C, et al. The art, science, and engineering of fuzzing: A survey[J]. IEEE Transactions on Software Engineering, 2021, 47(11):2312-2331.



- [23] BÖHME M, PHAM V T, ROYCHOUDHURY A. Coverage-based greybox fuzzing as Markov chain[J]. IEEE Transactions on Software Engineering, 2019, 45(5):489-506.
- [24] GODEFROID P, KIEZUN A, LEVIN M Y. Grammar-based whitebox fuzzing[C]//Proceedings of the 29th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI). [S.l.: s.n.], 2008: 206-215.
- [25] MCKEEMAN W M. Differential testing for software[J]. Digital Technical Journal, 1998, 10(1):100-107.
- [26] SALTON G, BUCKLEY C. Term-weighting approaches in automatic text retrieval[J]. Information Processing & Management, 1988, 24(5):513-523.
- [27] REIMERS N, GUREVYCH I. Sentence-BERT: Sentence embeddings using siamese BERT-networks[C]//Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP). [S.l.: s.n.], 2019: 3982-3992.
- [28] WANG W, WEI F, DONG L, et al. MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers[C]//Advances in Neural Information Processing Systems 33 (NeurIPS). [S.l.: s.n.], 2020: 5776-5788.
- [29] AUMÜLLER M, BERNHARDSSON E, FAITHFULL A. ANN-Benchmarks: A benchmarking tool for approximate nearest neighbor algorithms[C]//Proceedings of the International Conference on Similarity Search and Applications (SISAP). [S.l.: s.n.], 2020: 34-49.
- [30] WANG J, YI X, GUO R, et al. Milvus: A purpose-built vector data management system[C]//Proceedings of the 2021 International Conference on Management of Data (SIGMOD). [S.l.: s.n.], 2021: 2614-2627.
- [31] JÉGOU H, DOUZE M, SCHMID C. Product quantization for nearest neighbor search[J]. IEEE Transactions on Pattern Analysis and Machine Intelligence, 2011, 33(1):117-128.
- [32] JOLLIFFE I T. Principal component analysis[M]. 2nd ed. [S.l.]: Springer, 2002.
- [33] ERNST M D, PERKINS J H, GUO P J, et al. The Daikon system for dynamic detection of likely invariants[J]. Science of Computer Programming, 2007, 69(1-3):35-45.
- [34] GAO Y, XIONG Y, GAO X, et al. Retrieval-augmented generation for large language models: A survey[J]. arXiv preprint arXiv:2312.10997, 2024.
- [35] SHUSTER K, POFF S, CHEN M, et al. Retrieval augmentation reduces hallucination



in conversation[C]//Findings of the Association for Computational Linguistics: EMNLP 2021. [S.l.: s.n.], 2021: 3784-3803.

- [36] ZHU H, HALL P A V, MAY J H R. Software unit test coverage and adequacy[J]. ACM Computing Surveys, 1997, 29(4):366-427.
- [37] Free Software Foundation. Gcov — a test coverage program[Z]. [S.l.: s.n.], 2023.
- [38] OBERPARLEITER P. LCOV — the LTP GCOV extension[EB/OL]. 2023. <https://github.com/linux-test-project/lcov>.